

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

BÀI 8 Thiết kế Chi tiết

Thiết kế Class và Method

hung.letan@hust.edu.vn

0913030731

ONE LOVE. ONE FUTURE.

Nội dung

- Nắm vững các khái niệm nền tảng của thiết kế hướng đối tượng:
  - Lớp (Class), Đối tượng (Object), Thuộc tính (Attribute), Phương thức (Method).
- Tìm hiểu về các tiêu chí thiết kế tốt:
  - Coupling, Cohesion và Connascence.
- Hiểu rõ sự chuyển đổi từ Phân tích lớp (Analysis) sang Thiết kế (Design) lớp.
  - Quy trình tinh chỉnh cho việc chuyển
  - Tái sử dụng lớp : libraries, frameworks, and components.
  - tái cấu trúc , và tối ưu hóa thiết kế đối tượng.
- Hiểu cách đặc tả một lớp và phương thức của nó một cách chi tiết.
  - Biết cách xác định các hoạt động thiết kế chính và vai trò của constraints and contracts

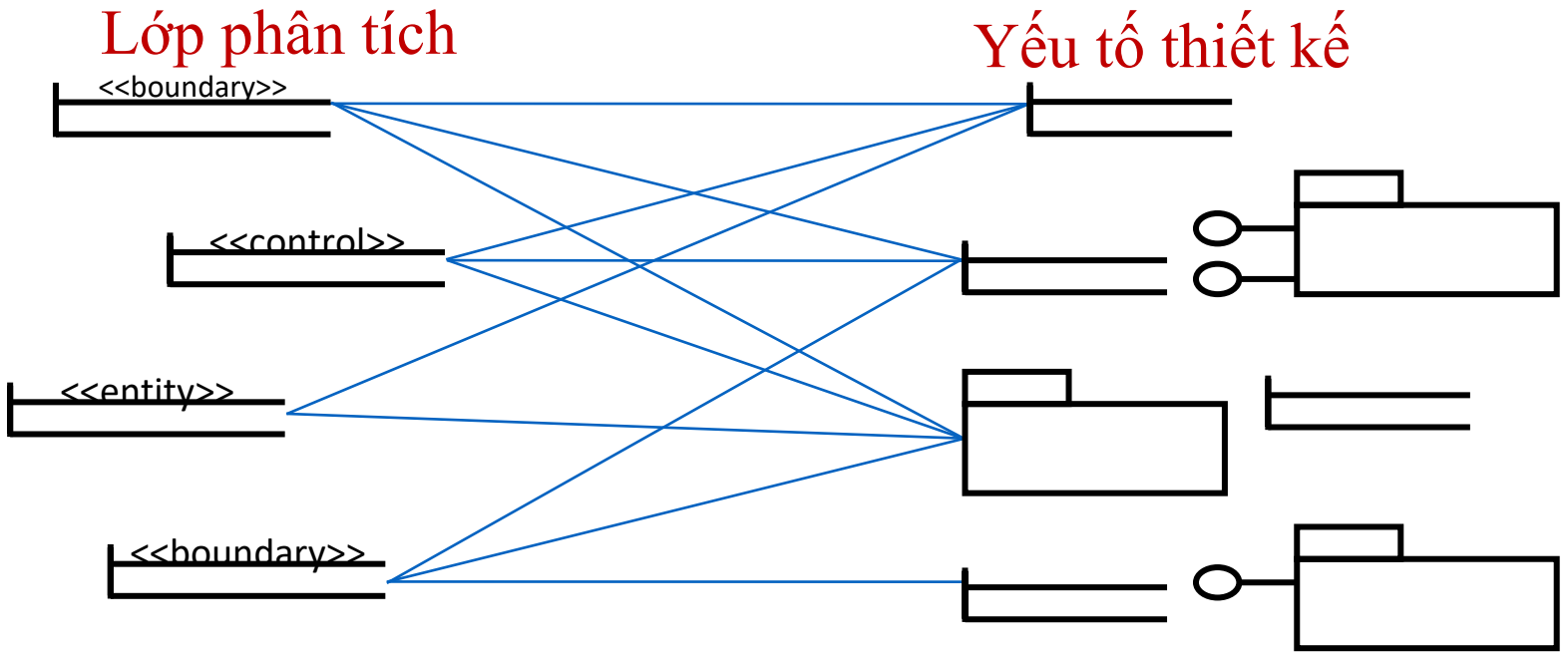
Các hoạt động chính trong Thiết kế

- 1. Thiết kế Kiến trúc (Architectural Design):
- 2. Thiết kế Lớp và Phương thức (Class and Method Design):
  - Nguyên tắc logic liên kết
  - Tinh chỉnh Thiết kế đối tượng
  - Tinh chỉnh Đặc tả Phương thức
- 3. Thiết kế Giao diện Người dùng (User Interface Design):
- 4. Thiết kế Dữ liệu (Data Design):

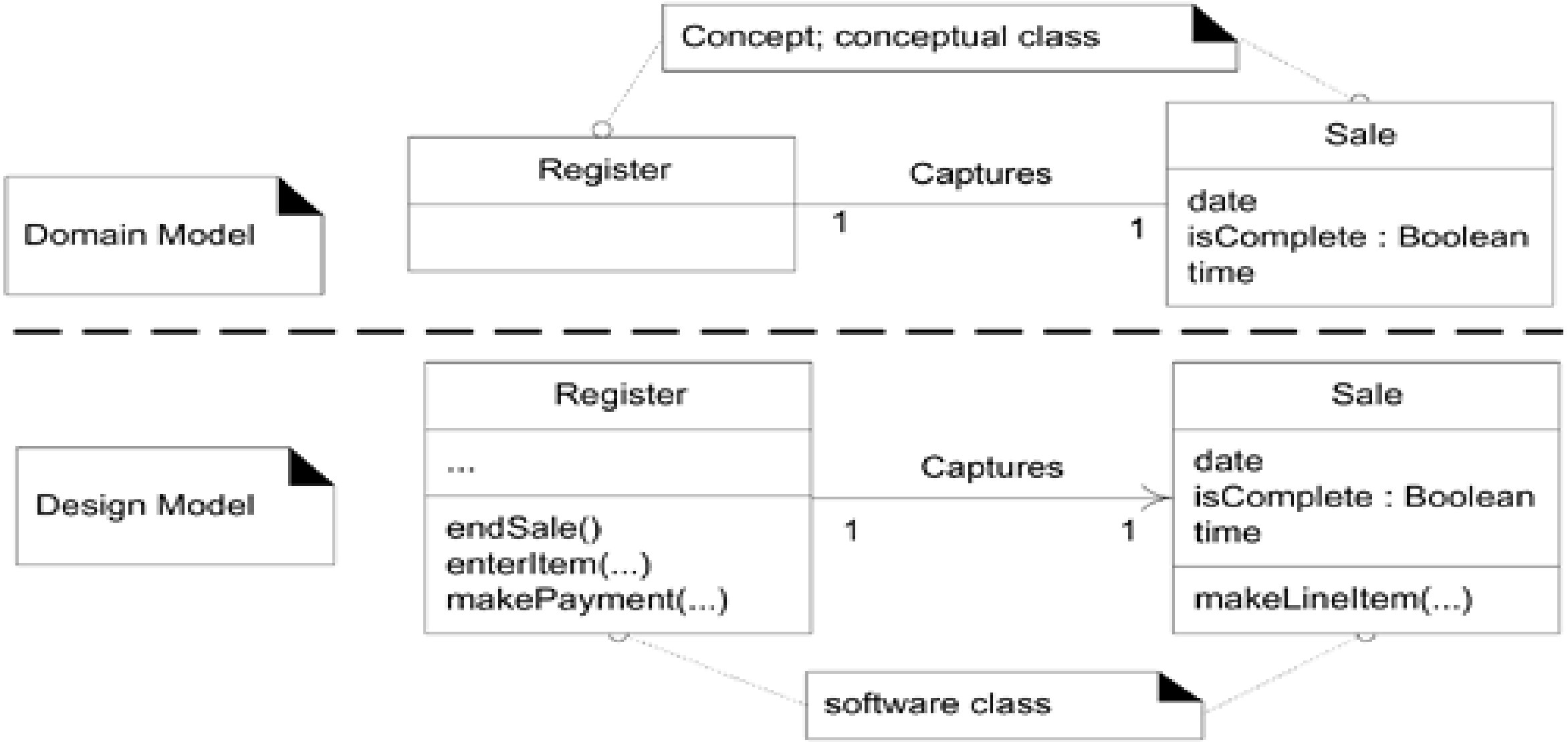
Vai trò của Lớp (Class) và Phương thức (Method)

- Thiết kế hướng đối tượng (Object-Oriented Design - OOD) coi
  - hệ thống là một tập hợp các đối tượng tương tác với nhau.
- Lớp (Class):
  - Là "khuôn mẫu" (template) định nghĩa cấu trúc (thuộc tính) và hành vi (phương thức) cho các đối tượng.
  - Đây là khối xây dựng cơ bản trong thiết kế.
- Phương thức (Method):
  - Là nơi chứa logic ứng dụng.
  - Thiết kế chi tiết các phương thức là công việc quan trọng nhất của pha thiết kế, đảm bảo hệ thống hoạt động đúng như yêu cầu
- Sản phẩm pha thiết kế:
  - Đặc tả thiết kế hệ thống (System Design Specification)

- Trong quá trình thiết kế, thiết kế viên tiến hành ánh xạ các lớp phân tích sang các yếu tố thiết kế



Many-to-Many Mapping


Concept; conceptual class
Domain Model
Register
Captures
1
1
Sale
date
isComplete : Boolean
time
Design Model
Register
...
endSale()
enterItem(...)
makePayment(...)
Captures
1
1
Sale
date
isComplete : Boolean
time
makeLineItem(...)
software class

Chuyển đổi từ Mô hình Phân tích sang Mô hình Thiết kế

- Tinh chỉnh và bổ sung chi tiết vào các mô hình phân tích đã có để chúng sẵn sàng cho việc lập trình.

- Hoạt động chính:
  1. Factoring (Tách nhân tố):
     - Tách một module phức tạp thành các module đơn giản hơn
     - (ví dụ: tách các thuộc tính chung của các lớp Nurse, Doctor ra một lớp cha Employee).
  2. Partitions and Collaborations (Phân vùng và Cộng tác):
     - Nhóm các lớp có tương tác cao với nhau thành các phân vùng hoặc gói (package).
  3. Layers (Các tầng):
     - Tổ chức thiết kế thành các tầng kiến trúc riêng biệt.
- Các khái niệm "Factoring", "Partitions", và "Layers" được giới thiệu như những kỹ thuật đầu tiên để cấu trúc lại và tinh chỉnh mô hình,
  - giúp hiểu rằng thiết kế không phải là tạo mới hoàn toàn mà là một quá trình tiến hóa từ những gì đã có.

NGUYÊN TẮC thiết kế tốt

- Vấn đề trung tâm của thiết kế phần mềm: một thiết kế không chỉ cần "chạy được" mà còn phải "tốt"
  - Thiết kế tốt: Dễ hiểu, dễ bảo trì, dễ mở rộng và tái sử dụng.
  - Thiết kế xấu: Phức tạp, khó hiểu, một thay đổi nhỏ có thể gây ra lỗi ở nhiều nơi khác.
- Làm thế nào để biết là tốt ?
  - Một thiết kế tốt là thiết kế cân bằng các yếu tố để giảm thiểu chi phí tổng thể của hệ thống trong suốt vòng đời của nó
  - Một số nguyên lý thiết kế
    - SOLID
    - DRY – Don’t repeat yourself
      - Tránh lặp lại trong thiết kế
    - KISS – Keep it simple, stupid!
      - Giữ các yếu tố thiết kế đơn giản
    - YAGNI – You ain’t gonna need it
      - Chỉ thiết kế khi cần

Nguyên lý SOLID

- SOLID
  - Single responsibility
  - Open/Close
  - Liskov substitution
  - Interface Segregation
  - Dependency Inversion

1. Single responsibility

- Trách nhiệm duy nhất
- Một lớp chỉ nên phục vụ cho một mục đích
  - Trong quá trình phân tích, lớp điều khiển nhận nhiều trách nhiệm khác nhau từ lớp biên.
  - Khi đó cần tách lớp điều khiển thành nhiều nhất để mỗi lớp chỉ đáp ứng cho một hoặc một nhóm trách nhiệm có liên quan mật thiết với nhau.
  - VD.
    - Lớp Đơn hàng chỉ nên quản lý các trách nhiệm liên quan tới những chi tiết đơn hàng chứ không nên nhận trách nhiệm tính toán giá trị chiết khấu/giảm giá cho nó.

Nguyên lý SOLID

- 2. Open/close
  - Mở cho mở rộng nhưng đóng cho thay đổi
  - Sử dụng kế thừa để mở rộng, tránh thay đổi lớp đã thiết kế
  - Cố gắng khái quát hóa cao để dễ dàng mở rộng, từ đó tránh phải thay đổi thiết kế.
    - VD. Thay vì viết một phương thức in thông tin đối tượng ra màn hình, hay viết phương thức để xuất thông tin đối tượng ra một kênh ra, được cung cấp dưới dạng tham số.

Nguyên lý SOLID

- 3. Liskov substitution
  - Bất cứ cá thể nào của lớp cha cũng có thể được thay thế bằng một cá thể của lớp con mà không ảnh hưởng đến tính đúng đắn
  - Vi phạm thường xảy ra khi thiết kế lớp con với phương thức ghi đè lên phương thức lớp cha nhưng lại không tương thích với lớp cha.
    - VD.
    - Hình chữ nhật có hai phương thức setHeight() và setWidth() để đặt chiều dài và chiều rộng một cách độc lập.
    - Hình vuông kế thừa từ hình chữ nhật nhưng phương thức setHeight hay setWidth của hình vuông thay đổi cả hai kích thước, từ đó có thể dẫn đến những sai lầm trong quản lý hình vẽ như kết quả zoom một tập đối tượng không mong đợi.

Nguyên lý SOLID

- Interface Segregation
  - Phân tách giao diện
  - Nên chia giao diện lớn thành nhiều giao diện nhỏ tương ứng với các mục đích khác nhau.
- Dependency inversion
  - Các lớp tầng trên không nên phụ thuộc trực tiếp vào các lớp tầng dưới.
    - Sử dụng giao diện
  - Lớp trừu tượng không nên phụ thuộc vào lớp chi tiết, các lớp nên giao tiếp với nhau qua giao diện

nguyên tắc thiết kế cho kiến trúc hiệu quả

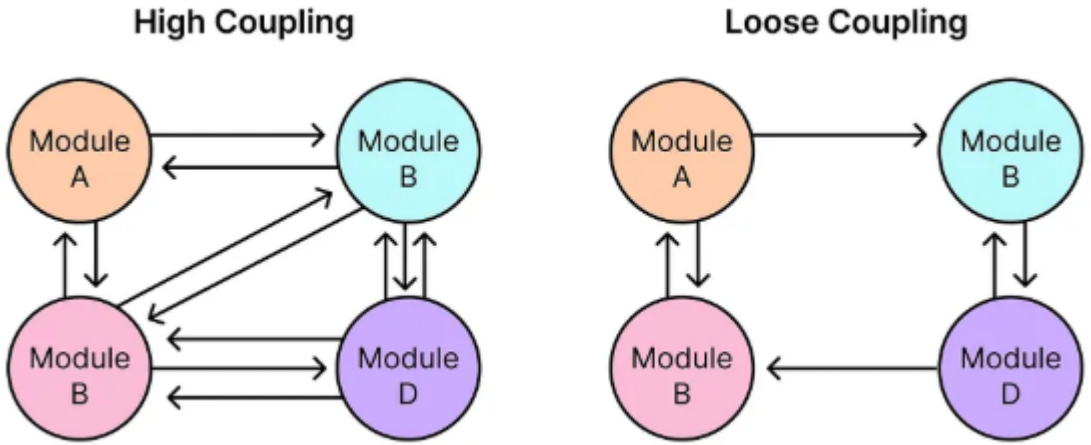
- Làm thế nào để đo lường chất lượng của một thiết kế?
- Ba tiêu chí quan trọng được dùng để đánh giá chất lượng của một thiết kế:
- The foundational measurements of structural software quality
  1. **Khớp nối (Coupling),**
  2. **Tính gắn kết (Cohesion)**
  3. **Connascence.**

Nguyên tắc Liên kết
Coupling, Cohesion,Connascence

1.Tiêu chí Coupling (Khớp nối)

- 'Coupling' - liên kết, đo lường mức độ phụ thuộc giữa các lớp (module)
  - "Khi xây dựng một hệ thống phần mềm, chúng ta không chỉ tạo ra các lớp riêng lẻ mà còn phải kết nối chúng lại với nhau.
  - Một hệ thống tốt giống như một đội làm việc hiệu quả, mỗi thành viên có thể làm việc độc lập mà không quá phụ thuộc vào người khác.
  - Sự phụ thuộc càng cao, thay đổi trong một phần của thiết kế gây ra những thay đổi trong các phần khác

là thước đo mức độ liên kết giữa hai thành phần (thói quen, phương thức, hàm, lớp, mô-đun, v.v.).

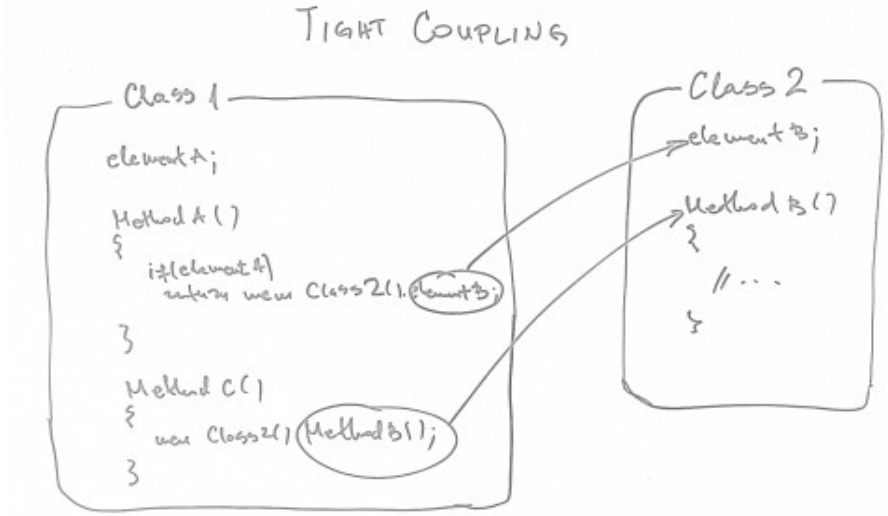


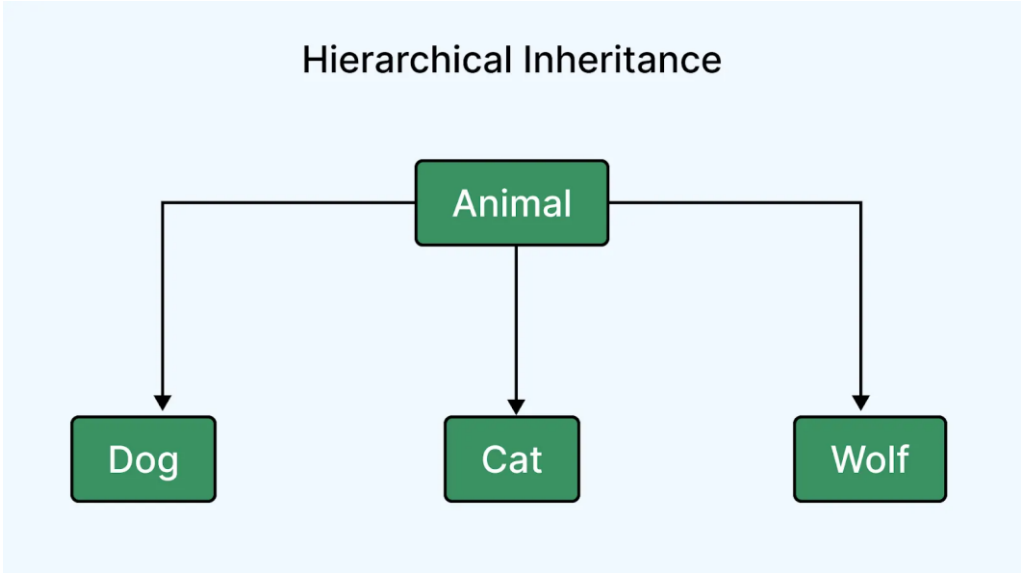
Low Coupling 'liên kết lỏng

- Tính mô-đun (Modularity)
  - Tính mô-đun là nguyên tắc thiết kế hệ thống phần mềm được chia nhỏ thành các đơn vị độc lập và khép kín, gọi là mô-đun.
  - Mỗi mô-đun đảm nhận một chức năng cụ thể và rõ ràng
- Khả năng tái sử dụng (Reusability)
  - mô-đun thực hiện một chức năng cụ thể và độc lập, nên dễ dàng được "lấy ra" và sử dụng lại
- Khả năng bảo trì (Maintainability)
  - thay đổi một thành phần mà không làm hỏng các thành phần khác,
- Khả năng kiểm thử (Testability)
  - mô-đun độc lập được kiểm thử riêng lẻ (kiểm thử đơn vị) VÀ riêng biệt, giúp đơn giản hóa quá trình kiểm thử.

Các loại liên kết

- Đối với hệ thống OO, Coad và Yourdon* đã xác định hai loại khớp nối cần xem xét:
  1. Interaction Coupling (liên kết qua tưởng tác/thông điệp)
  2. Inheritance Coupling (liên kết qua kế thừa)
- Việc hiểu và cân bằng hai loại khớp nối này l
  - Là một kỹ năng quan trọng để tạo ra một thiết kế hướng đối tượng tốt, dễ bảo trì và mở rộng.





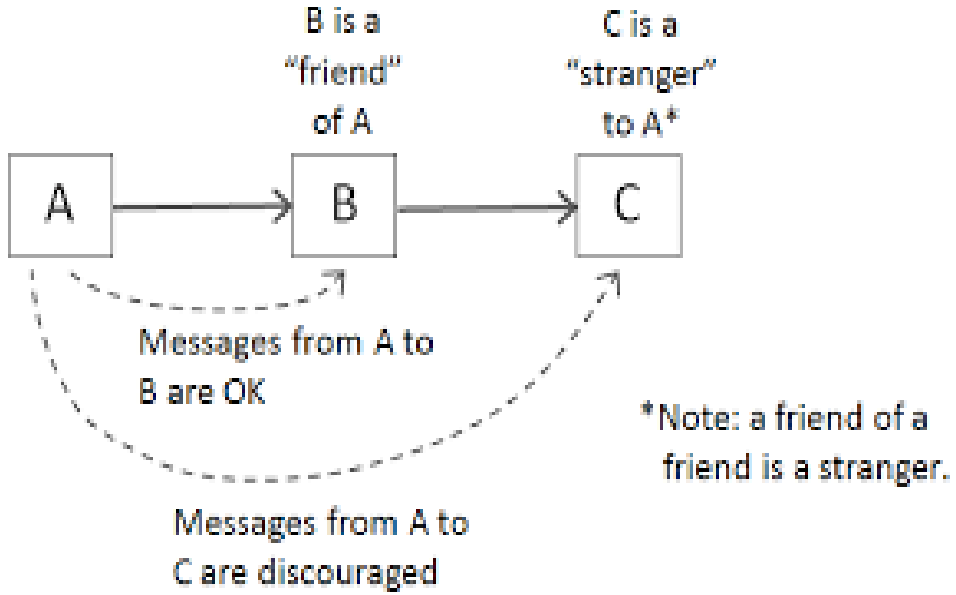
Interaction Coupling (Khớp nối Tương tác)

- Khớp nối tương tác
  - đề cập đến sự phụ thuộc lẫn nhau giữa các phương thức (methods) và đối tượng (objects) thông qua việc truyền thông điệp (message passing).
  - Đây là loại khớp nối phổ biến nhất trong các hệ thống hướng đối tượng, xảy ra khi một đối tượng này gọi một phương thức (dịch vụ) của một đối tượng khác.
  - Mức độ của khớp nối này phụ thuộc vào mức độ phức tạp của các thông điệp được truyền đi.
  - Mục tiêu là giảm thiểu khớp nối tương tác.

  - DON'T TALK TO STRANGERS

Quy luật Demeter (Law of Demeter [Lieberherr & Holland 89])

- Để đạt được mục tiêu liên kết lỏng lẻo,
  - quy tắc hữu ích gọi là Quy luật Demeter, hay còn gọi là 'nguyên tắc giảm quan hệ'.
  - Ý tưởng: chỉ nói chuyện với những người thân,
    - thay vì cố gắng nói chuyện với bạn của bạn .
- Quy luật Demeter : Một đối tượng chỉ nên gửi thông điệp đến:
  - 1. Chính nó (itself).
  - 2. Một đối tượng chứa trong thuộc tính của nó.
  - 3. Một đối tượng được truyền vào như một tham số của phương thức.
  - 4. Một đối tượng được tạo bởi chính phương thức đó.
- Mục tiêu
  - là hạn chế việc một đối tượng "biết" về cấu trúc bên trong của các đối tượng khác.



DON'T TALK TO STRANGERS

Inheritance Coupling (Khớp nối Kế thừa)

- Khớp nối kế thừa
  - đề cập đến mức độ liên kết chặt chẽ giữa các lớp trong một hệ thống phân cấp kế thừa (inheritance hierarchy).
  - Đây là mối quan hệ giữa một lớp con (subclass) và lớp cha (superclass) của nó.
- Liên kết kế thừa thấp
  - xảy ra khi một lớp con không kế thừa nhiều tính năng không liên quan hoặc không sử dụng,
- Liên kết cao
  - Các vấn đề có thể phát sinh khi một lớp con có thể truy cập trực tiếp vào các thuộc tính hoặc phương thức được che giấu (hidden) của lớp cha, vi phạm nguyên tắc đóng gói và che giấu thông tin.

Xử lý vấn đề ghép nối kế thừa

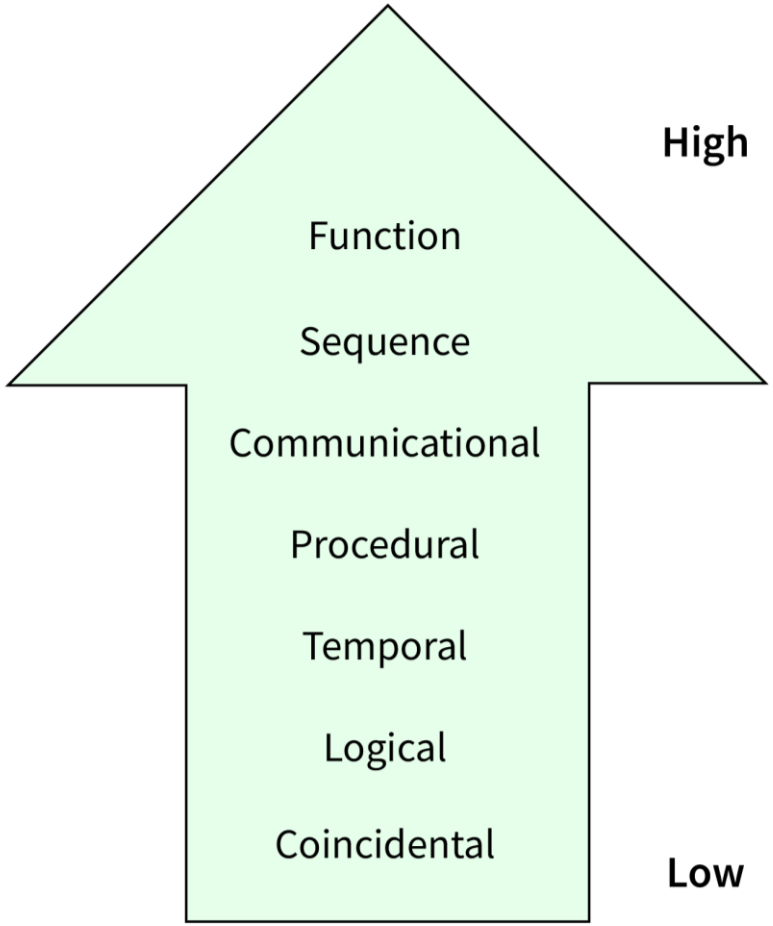
- Ưu tiên thành phần hơn là thừa kế
  - Kế thừa là mối quan hệ "là-một": A Carlà một Vehicle.
  - Composition là mối quan hệ "có-một": A Carcó một Engine.
    - Sử dụng composition thường dẫn đến sự kết hợp lỏng lẻo hơn
- Use interfaces for contracts
  - Thay vì kế thừa từ một lớp cha cụ thể, một lớp có thể triển khai một giao diện định nghĩa một hợp đồng cho hành vi
    - Ví dụ: Một CreditCardPaymentlớp và một PayPalPaymentlớp khác đều có thể triển khai một PaymentGateway giao diện.
- Sử dụng lớp cơ sở trừu tượng:
  - Lớp trừu tượng có thể cung cấp một mẫu cho các lớp con trong khi vẫn cho phép có một mức độ linh hoạt nhất định,

Tiêu chí 2: Sự gắn kết (Cohesion

- Cohesion
  - Đo lường mức độ "tập trung" hay "đơn nhiệm" của một module.
  - "Nếu 'Coupling' nói về mối quan hệ giữa các lớp, thì 'Cohesion' hay sự gắn kết, lại tập trung vào bên trong mỗi lớp.
- Gắn kết cao (High Cohesion).
  - Một lớp có tính gắn kết cao sẽ chỉ thực hiện một nhiệm vụ duy nhất, giống như một chuyên gia chỉ giỏi về một lĩnh vực
  - gắn kết cao' (High Cohesion) lại là một tiêu chí quan trọng cho một thiết kế tốt, giúp hệ thống dễ hiểu và dễ bảo trì hơn."
  - Ví dụ
    - Một lớp chỉ nên đại diện cho một khái niệm duy nhất.
    - Một phương thức chỉ nên thực hiện một nhiệm vụ duy nhất.

Cohesion Type

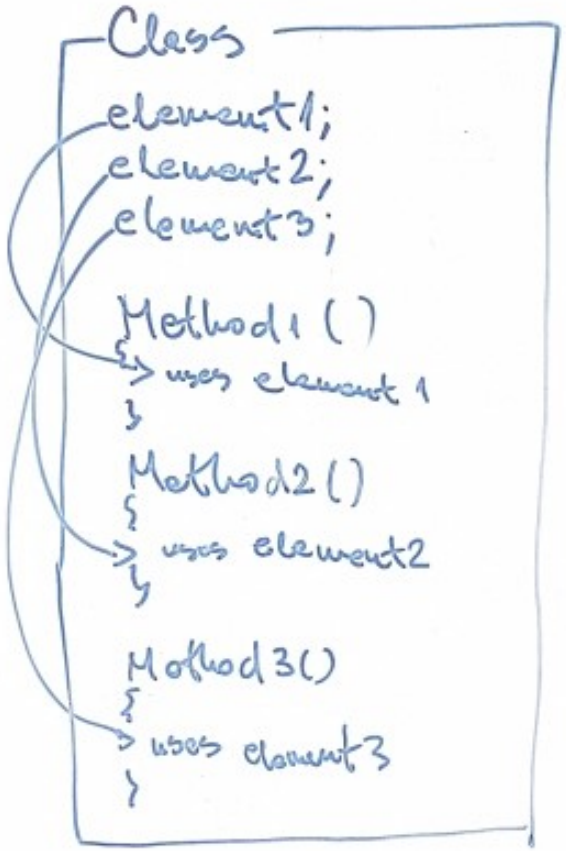
1. Sự gắn kết ngẫu nhiên (Coincidental cohesion) – *Bad* Very low cohesion
2. Sự gắn kết logic (Logical cohesion)
3. SỰ GẮN KẾT THỜI GIAN (TEMPORAL COHESION)
4. Sự gắn kết theo thủ tục (Procedural cohesion)
5. Sự gắn kết thông tin / trao đổi (Communicational/informational cohesion)
6. Sự liên kết tuần tự (Sequential cohesion)
7. Sự gắn kết về mặt chức năng (Functional cohesion) *Good*



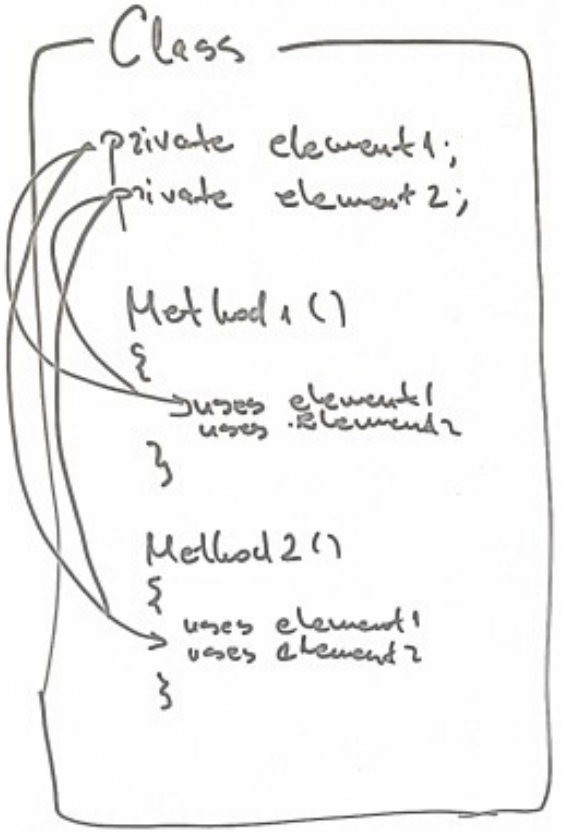
Ba loại liên kết

- Ba loại liên kết chung đã được Coad và Yourdon* xác định cho các hệ thống OO:
  1. Method Cohesion:
     - Một phương thức chỉ nên thực hiện một chức năng duy nhất.
     - giải quyết sự gắn kết trong một phương pháp riêng lẻ.
  2. Class Cohesion:
     - Một lớp chỉ nên đại diện cho một "thứ" duy nhất.
     - Một lớp chỉ nên có các thuộc tính và phương thức cần thiết
     - Tất cả các thuộc tính và phương thức phải cần thiết để định nghĩa đầy đủ cho "thứ" đó.
  3. Generalization/Specialization Cohesion:
     - Hệ thống phân cấp kế thừa chỉ nên thể hiện mối quan hệ "is-a-kind-of"

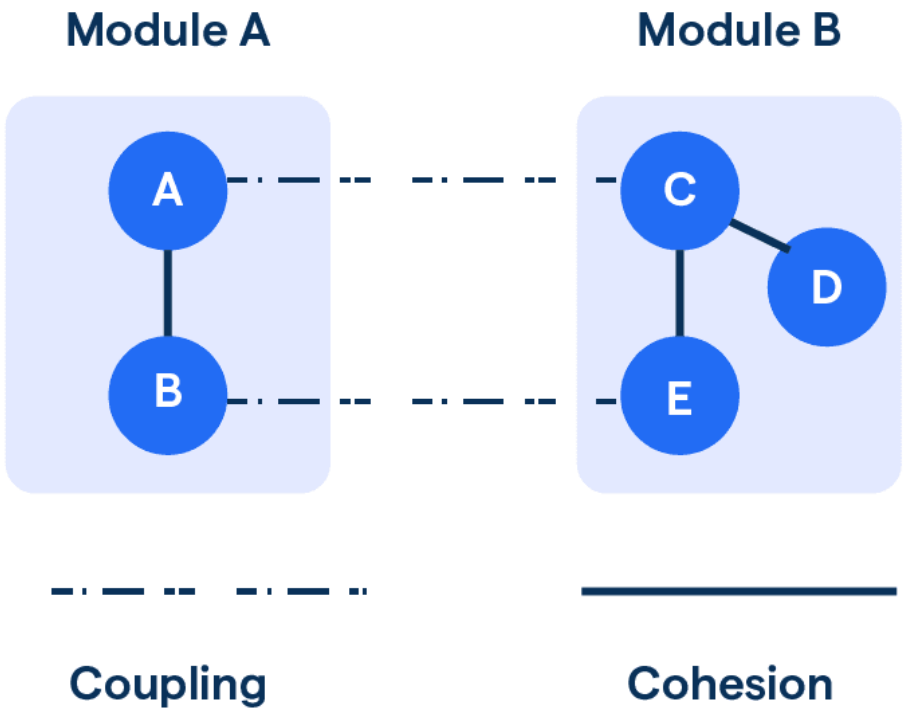
Low Cohesion



High Cohesion



COHESION	COUPLING
Cohesion thể hiện mối quan hệ bên trong module.	Coupling thể hiển sự liên kết giữa các modules.
Cohesion thể hiện sức mạnh liên kết giữa các chức năng	Coupling thể hiện quan hệ phụ thuộc vào nhiều modules.
Cohesion đánh giá chất lượng mà một component / module tập trung vào một việc đơn lẻ.	Coupling là đánh giá mức độ một component / module liên kết với mudule khác.

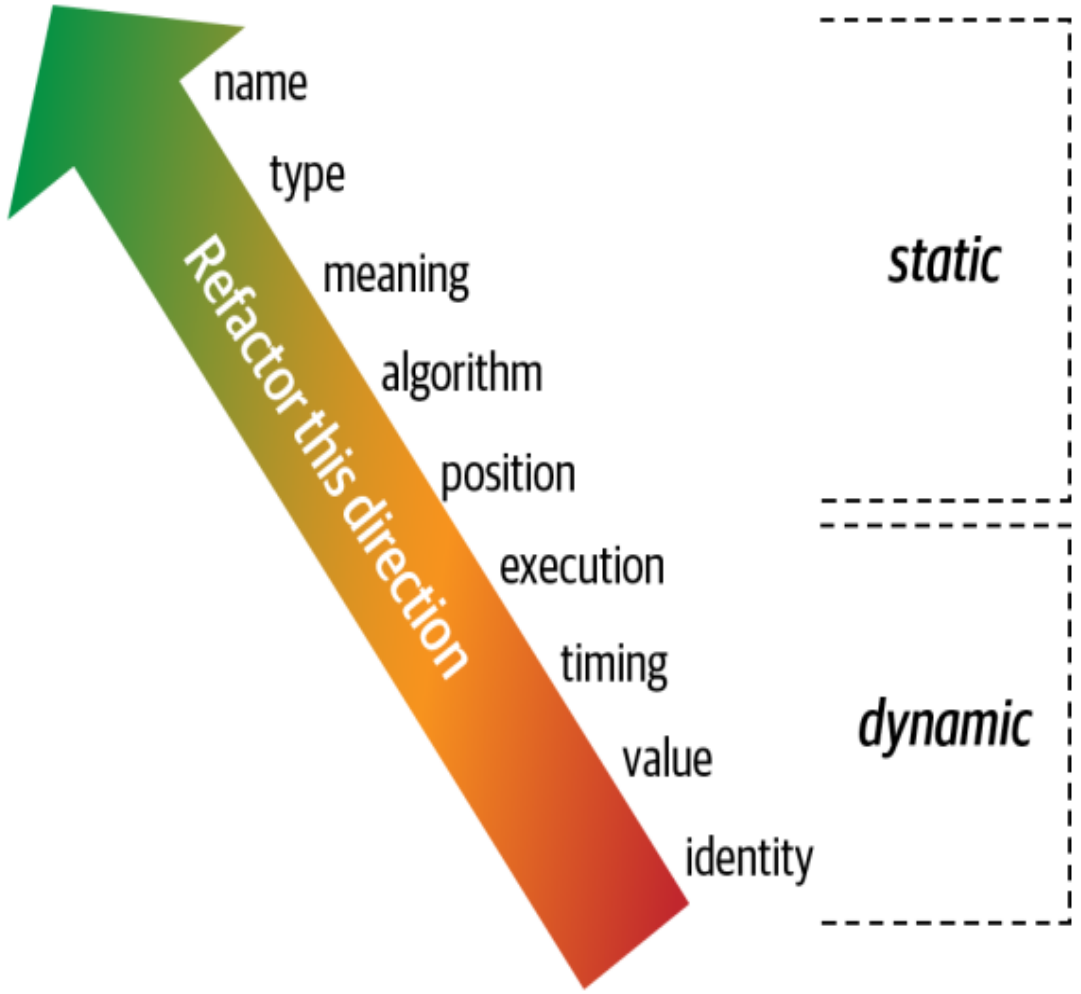


Tiêu chí 3: Connascence (Tính liên kết)

- Connascence (Tính liên kết):
  - Một khái niệm tổng quát hơn của Coupling và Cohesion.
  - Hai thành phần được gọi là có liên kết (connascent) nếu một thay đổi ở một thành phần đòi hỏi phải thay đổi ở thành phần kia để duy trì tính đúng đắn của hệ thống.
- Connascence giúp các nhà thiết kế và phát triển phần mềm xác định và cải thiện chất lượng mô-đun của mã nguồn
- Mục tiêu:
  - Tối thiểu hóa connascence giữa các ranh giới đóng gói và tối đa hóa connascence bên trong một ranh giới đóng gói.

Phân loại Connascence

- Static Connascence tĩnh
  - là sự liên kết ở cấp độ mã nguồn: Name, Type, Convention, Algorithm, Position (Static)
  - được xác định bằng cách phân tích mã nguồn tĩnh.
- Dynamic Connascence động
  - là sự liên kết ở cấp độ thời gian chạy (runtime) Execution, Timing, Values, Identity (Dynamic).
  - khó xác định hơn vì nó đòi hỏi sự hiểu biết về thực thi một ứng dụng cụ thể.
- Các kiến trúc sư
  - ưu tiên connascence tĩnh
  - vì các công cụ hiện đại giúp việc tái cấu trúc (refactor) các liên kết tĩnh dễ dàng hơn



Connascence Tĩnh (Static Connascence)

- Connascence of Name (Liên kết về Tên - CoN):
  - Nhiều thành phần phải thống nhất về tên của một thực thể.
  - Đây là dạng connascence yếu nhất và được ưa thích nhất..
- Connascence of Type (Liên kết về Kiểu - CoT):
  - Nhiều thành phần phải thống nhất về kiểu của một thực thể.
- Connascence of Meaning or Convention (Liên kết về Ý nghĩa hoặc Quy ước - CoM/CoC):
  - một lớp có một thuộc tính trong đó một dải giá trị có ý nghĩa ngữ nghĩa riêng
  - (ví dụ: số tài khoản từ 1000 đến 1999 là tài sản).
- Connascence of Position (Liên kết về Vị trí - CoP):
  - Các thành phần phải thống nhất về thứ tự của các giá trị.
- Connascence of Algorithm (Liên kết về Thuật toán - CoA):
  - Nhiều thành phần phải thống nhất về một thuật toán cụ thể.

Connascence Động (Dynamic Connascence)

- Connascence of Execution (Liên kết về Thực thi - CoE):
  - Ví dụ, một số thuộc tính của một đối tượng phải được thiết lập theo một thứ tự nhất định trước khi một phương thức được gọi.
- Connascence of Timing (Liên kết về Thời gian - CoT):
  - Ví dụ phổ biến là tình trạng tranh chấp (race condition) do hai luồng (thread) thực thi cùng lúc, ảnh hưởng đến kết quả của hoạt động chung.
- Connascence of Values (Liên kết về Giá trị - CoV):
  - Xảy ra khi nhiều giá trị liên quan đến nhau và phải thay đổi cùng nhau.
- Connascence of Identity (Liên kết về Định danh - CoI):
  - Xảy ra khi nhiều thành phần phải tham chiếu đến cùng một thực thể.

Các Quy tắc Hướng dẫn về Connascence

- Ba nguyên tắc về connascence của Meilir Page-Jones
  - là những hướng dẫn cơ bản trong kỹ thuật phần mềm nhằm cải thiện khả năng bảo trì và giảm thiểu sự phụ thuộc phức tạp giữa các thành phần mã nguồn
  - 1. Giảm thiểu connascence tổng thể
    - Nguyên tắc giảm thiểu tổng thể sự phụ thuộc trong toàn bộ hệ thống bằng cách chia nó thành các thành phần hoặc mô-đun được đóng gói chặt chẽ.
  - 2. Giảm thiểu connascence xuyên qua các ranh giới đóng gói
    - Ngay cả khi đã chia hệ thống thành các mô-đun riêng biệt, vẫn có những sự phụ thuộc.
    - Nguyên tắc giảm thiểu mọi sự phụ thuộc (connascence) sót lại giữa các thành phần đã được đóng gói.
  - 3. Tối đa hóa connascence bên trong các ranh giới đóng gói
    - Nguyên tắc khuyến khích tăng cường sự phụ thuộc (connascence) bên trong một thành phần đóng gói, hoạt động cùng nhau để thực hiện một mục đích chung

Tổng quan các Hoạt động Thiết kế Đối tượng

- "Sau khi nắm vững các nguyên tắc,
  - chúng ta sẽ đi vào các hoạt động cụ thể của việc thiết kế lớp.
- Đây là một quá trình lặp đi lặp lại,
  - bổ sung chi tiết, tìm kiếm cơ hội tái sử dụng, tái cấu trúc và tối ưu hóa bản thiết kế.
- tinh chỉnh một lớp phân tích thành một lớp thiết kế.
  - bổ sung các chi tiết kỹ thuật cần thiết để chuẩn bị cho việc lập trình
- Quy trình
  1. Bổ sung Đặc tả,
  2. Xác định Cơ hội Tái sử dụng,
  3. Tái cấu trúc Thiết kế,
  4. Tối ưu hóa Thiết kế

Hoạt động 1 :
Adding Specifications

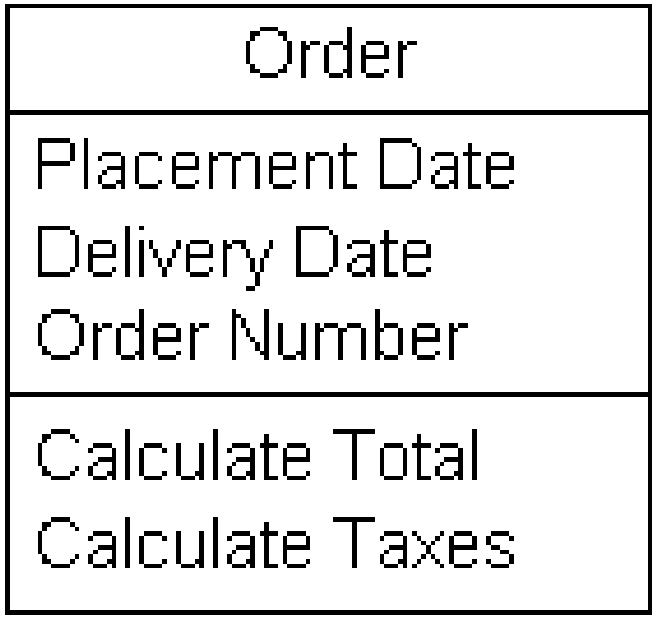
Hoạt động 1: Bổ sung các Đặc tả (Adding Specifications)

- 'Bổ sung các Đặc tả'.
  - rà soát lại các mô hình, xác định rõ ràng phạm vi truy cập cho từng thuộc tính và phương thức, và định nghĩa 'chữ ký' (signature) cho mỗi phương thức.
  - Bước này đảm bảo rằng mọi chi tiết cần thiết cho việc lập trình đều được xác định rõ ràng."
- Mục tiêu:
  - Biến các lớp phân tích thành các lớp thiết kế bằng cách thêm các chi tiết kỹ thuật.
- Các bước:
  1. Hoàn thiện Visibility: Xác định phạm vi truy cập (public (+), private (-), protected (#)).
  2. Định nghĩa Signature của phương thức: Gồm tên, tham số đầu vào và kiểu dữ liệu trả về.
  3. Xác định các ràng buộc (Constraints) và hợp đồng (Contracts).
  4. Xử lý các Vi phạm Ràng buộc (Handling Constraint Violations):

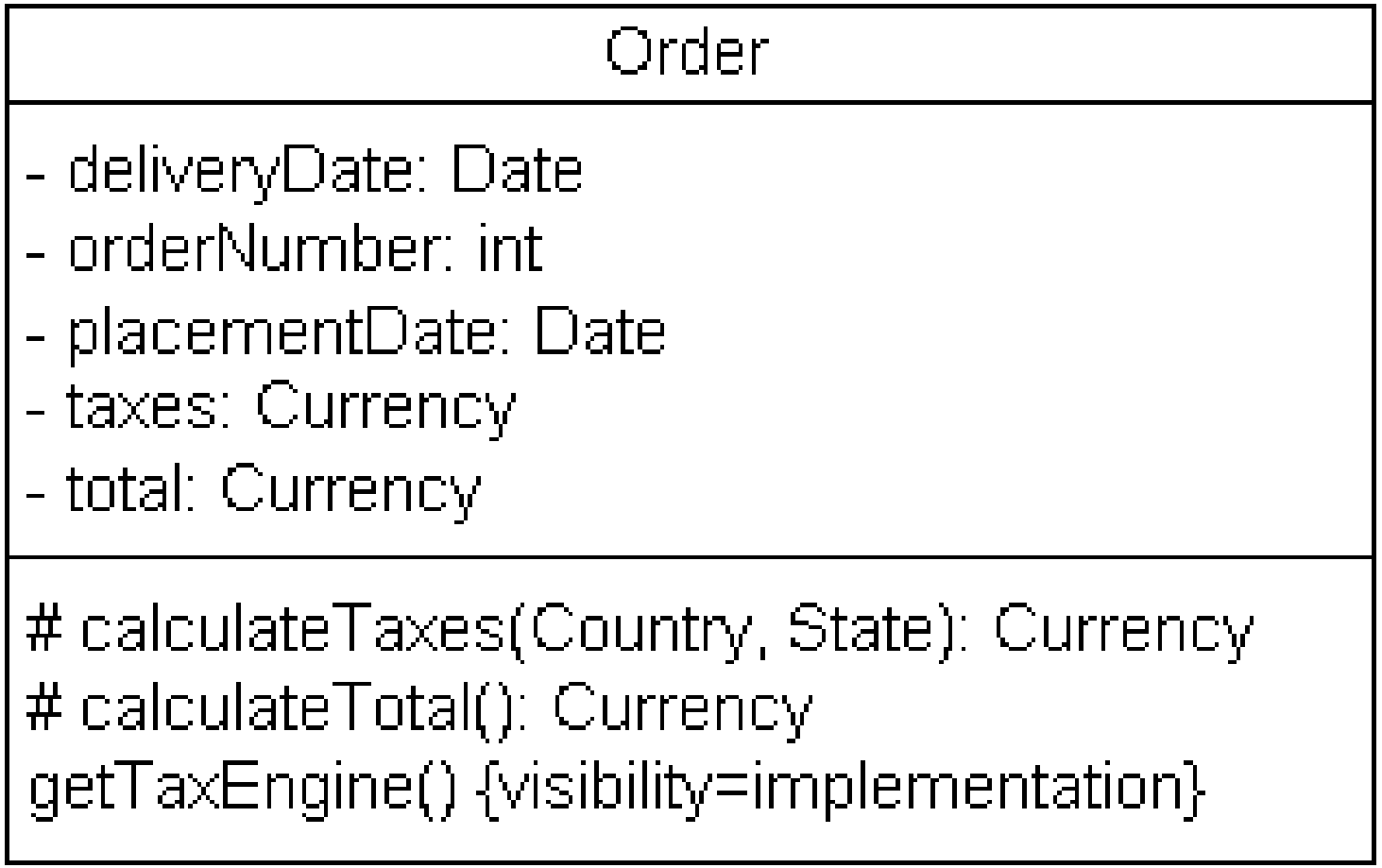
1.1 Hoàn thiện Khả năng hiển thị (Finalizing Visibility):

- quyết định khả năng hiển thị (visibility) của từng thuộc tính (attribute) và phương thức (method) trong mỗi lớp.
- Có ba mức hiển thị chính:
  - private (-): chỉ có thể được truy cập từ bên trong lớp đó.
  - public (+): có thể được truy cập từ bất kỳ lớp nào khác.
  - protected (#): được truy cập bởi chính lớp đó và các lớp con (subclasses)
- Việc xác định đúng mức hiển thị
  - là quan trọng để thực thi nguyên tắc che giấu thông tin,
  - giúp giảm thiểu khớp nối (coupling) và làm cho hệ thống dễ bảo trì hơn.

Analysis



Design



1.2 Xác định Chữ ký Phương thức (Defining Method Signatures):

- Chữ ký phương thức là sự kết hợp của tên phương thức và danh sách tham số gồm ba phần:
  1. Tên phương thức (Method Name):
  2. Danh sách tham số (Parameter List):
  3. Kiểu giá trị trả về (Return Type):
- Vd: withdraw(double amount):
  - **Method Name:** withdraw
  - **Parameter List:** double (one parameter of type double)
  - **Return Type:** boolean
  - **Full Signature:** withdraw(double)
- Việc định nghĩa chữ ký phương thức
  - giúp cụ thể hóa các thông điệp (messages) được truyền giữa các đối tượng, như đã được mô tả trong các sơ đồ tương tác (interaction diagrams).

1.3 Xác định các Ràng buộc (Defining Constraints):

- Hoạt động này xác định các quy tắc hoặc ràng buộc mà các đối tượng phải tuân thủ.
- Các ràng buộc đảm bảo tính toàn vẹn và đúng đắn của dữ liệu và hành vi hệ thống.
- Có ba loại ràng buộc chính được xác định:
  - Tiền điều kiện (Preconditions): ràng buộc phải được đáp ứng trước khi một phương thức có thể thực thi.
  - Hậu điều kiện (Postconditions): ràng buộc phải được đáp ứng sau khi một phương thức đã thực thi xong.
  - Bất biến (Invariants): ràng buộc phải luôn đúng đối với tất cả các thể hiện (instances) của một lớp tại bất kỳ thời điểm nào.
- Các ràng buộc này thường được định nghĩa chính thức bằng Ngôn ngữ Ràng buộc Đối tượng (OCL - Object Constraint Language) để đảm bảo tính chính xác và không mơ hồ.
- Chúng cũng là một phần quan trọng của Hợp đồng (Contracts) giữa các đối tượng.

•Một sinh viên không thể đăng ký quá 5 khóa học.
•Mỗi khóa học phải có tối thiểu 10 học viên và tối đa 50 học viên

1.4 Xử lý các Vi phạm Ràng buộc (Handling Constraint Violations):

- Nhà thiết kế không chỉ phải xác định các ràng buộc mà còn phải quyết định cách hệ thống sẽ xử lý khi một ràng buộc bị vi phạm.
  - Việc xử lý các vi phạm này được gọi là exceptions .
  - Việc thiết kế cách xử lý lỗi là một phần quan trọng, không nên để lại cho lập trình viên tự quyết định.
- Các lựa chọn xử lý bao gồm:
  - Hủy bỏ hoạt động (abort).
  - Tự động hoàn tác (undo) thay đổi gây ra vi phạm.
  - Cho phép người dùng cuối quyết định cách khắc phục vi phạm.
- Vd Hệ thống đăng ký đại học
  - Một sinh viên không thể đăng ký quá 5 khóa học.
  - Mỗi khóa học phải có tối thiểu 10 học viên và tối đa 50 học viên

Tóm lại,

- hoạt động "Adding Specifications" là
  - một bước thiết yếu để chuyển đổi các mô hình phân tích thành các đặc tả thiết kế chi tiết,
- cung cấp một "bản thiết kế" rõ ràng cho các lớp và phương thức,
- từ đó giúp lập trình viên xây dựng hệ thống một cách hiệu quả và chính xác

Hoạt động 2. Xác định Cơ hội Tái sử dụng

Hoạt động 2: Xác định Cơ hội Tái sử dụng (Reuse)

- Thay vì 'phát minh lại bánh xe' cho mỗi dự án mới,
  - chúng ta có thể tận dụng các giải pháp đã có.
- Các Phương pháp phổ dụng:
  1. Design Patterns (Mẫu thiết kế):
     - Các giải pháp đã được chứng minh cho các vấn đề thiết kế phổ biến.
  2. Frameworks:
     - Một tập hợp các lớp hợp tác cung cấp một kiến trúc có thể tái sử dụng.
  3. Class Libraries (Thư viện lớp):
     - Một tập hợp các lớp đã được triển khai, tập trung vào chức năng cụ thể.
  4. Components (Thành phần):
     - Các đơn vị phần mềm độc lập, có thể thay thế với các giao diện (API) được xác định rõ ràng.

2.1 Mẫu thiết kế (Design Patterns)

- Design Patterns
  - Là một nhóm các lớp cộng tác hữu ích cung cấp giải pháp cho một vấn đề thường gặp.
  - "là những 'bí quyết' đã được đúc kết từ kinh nghiệm của rất nhiều lập trình viên đi trước.
- Chúng là những giải pháp mẫu cho các vấn đề thiết kế thường gặp.
  - Việc áp dụng các mẫu này giúp giải quyết vấn đề hiệu quả
  - làm cho thiết kế của chúng ta trở nên chuẩn mực và dễ hiểu hơn.
- Các mẫu thiết kế khác nhau về độ phức tạp, mức độ chi tiết và khả năng áp dụng cho toàn bộ hệ thống đang được thiết kế.
- Design pattern giúp cung cấp các mô hình thiết kế phần mềm,
  - giúp code trở nên tinh gọn, linh hoạt và dễ quản lý hơn

Phân loại các mẫu

- Phân loại theo độ phức tạp
  - idioms.
    - Những mẫu cơ bản và cấp thấp nhất thường được gọi là thành ngữ .
    - Chúng thường chỉ áp dụng cho một ngôn ngữ lập trình duy nhất.
  - Architectural patterns
    - Các mẫu phổ biến và cấp cao nhất là các mẫu kiến trúc .
    - được sử dụng để thiết kế kiến trúc của toàn bộ ứng dụng.
- Phân loại theo mục đích
  1. Creational pattern
     - cung cấp cơ chế tạo đối tượng giúp tăng tính linh hoạt và khả năng tái sử dụng mã hiện có.
  2. Structural pattern
     - giải thích cách lắp ráp các đối tượng và lớp thành các cấu trúc lớn hơn,
  3. Behavioral patterns
     - đảm bảo giao tiếp hiệu quả và phân công trách nhiệm giữa các đối tượng.

Creational design pattern

- Creational design pattern
  - Là cách tạo đối tượng, giúp ẩn đi logic khởi tạo và cung cấp các phương thức linh hoạt.
  - giúp kiểm soát tốt hơn quá trình khởi tạo đối tượng và tăng khả năng tái sử dụng code.
- sử dụng?
  - Khi cần tạo đối tượng phức tạp, có nhiều thuộc tính.
  - Khi muốn đảm bảo tính nhất quán trong việc khởi tạo đối tượng.
- Ưu điểm:
  - Giúp tách biệt logic khởi tạo, tăng tính linh hoạt và dễ mở rộng.
  - Giảm sự phụ thuộc giữa các class, giúp code dễ bảo trì hơn.
- Nhược điểm:
  - Nếu lạm dụng có thể làm code phức tạp hơn, khó hiểu với người mới.
- Ví dụ các mẫu thiết kế Creational phổ biến:
  - Singleton: Đảm bảo một class chỉ có một thể hiện duy nhất. .
  - Factory method: Có thể tạo đối tượng mà không cần chỉ định lớp cụ thể.
  - Builder: Hỗ trợ tạo đối tượng có nhiều thuộc tính mà không cần constructor.

Structural design pattern

- Structural design pattern
  - Là cách tổ chức các lớp và đối tượng để tạo ra cấu trúc phần mềm linh hoạt.
  - Các mẫu này giúp đơn giản hóa cấu trúc code, tăng khả năng mở rộng và tái sử dụng.
- sử dụng?
  - Khi cần kết hợp nhiều lớp và đối tượng theo cách linh hoạt hơn.
  - Khi muốn đơn giản hóa cấu trúc hệ thống, tránh trùng lặp code.
- Ưu điểm:
  - Giúp tăng tính module hóa, code gọn gàng, dễ hiểu hơn.
  - Hỗ trợ tái sử dụng các thành phần trong hệ thống.
- Nhược điểm:
  - Đôi khi cấu trúc trở nên phức tạp nếu áp dụng không đúng cách.
- Ví dụ các mẫu thiết kế Structural phổ biến:
  - Adapter: Cho phép các class với giao diện khác nhau có thể làm việc cùng nhau.
  - Decorator: Thêm chức năng mới vào một đối tượng mà không thay đổi cấu trúc của nó.
  - Facade: Cung cấp một giao diện đơn giản để làm việc với hệ thống phức tạp.

Behavioral design pattern

- Behavioral design pattern
  - Là cách các đối tượng giao tiếp với nhau, giúp định nghĩa các hành vi phức tạp mà vẫn giữ cho hệ thống linh hoạt và dễ bảo trì.
- sử dụng?
  - Khi có nhiều đối tượng tương tác với nhau theo cách phức tạp.
  - Khi muốn định nghĩa rõ ràng hành vi giữa các thành phần trong phần mềm.
- Ưu điểm:
  - Giúp quản lý luồng giao tiếp giữa các đối tượng một cách hiệu quả.
  - Tăng tính linh hoạt, giúp hệ thống dễ mở rộng.
- Nhược điểm:
  - Nếu sử dụng quá nhiều có thể làm code khó đọc và bảo trì.
- Ví dụ các mẫu thiết kế Behavioral phổ biến:
  - Observer: Một đối tượng thay đổi sẽ thông báo cho các đối tượng liên quan.
  - Strategy: Cho phép thay đổi thuật toán thực thi mà không cần sửa đổi code gốc.
  - Command: Biến một thao tác thành một đối tượng, giúp dễ dàng quản lý và mở rộng.

2.2 Frameworks

- Một framework
  - là một tập hợp các lớp đã được triển khai, có thể được sử dụng làm cơ sở để hiện thực hóa một ứng dụng.
- Các framework thường chuyên biệt cho một lĩnh vực cụ thể
  - (ví dụ: framework cho ứng dụng thương mại điện tử) và cung cấp một bộ khung các lớp cộng tác để xây dựng ứng dụng nhanh hơn.
- Khung quan trọng trong việc thúc đẩy tái sử dụng trong thiết kế lớp
  - cung cấp kiến trúc được xác định trước và một tập hợp các lớp cộng tác có thể được mở rộng và tùy chỉnh.
- Cách tiếp cận này tạo điều kiện cho việc tái sử dụng ở mức độ chi tiết cao hơn so với từng lớp hoặc thành phần riêng lẻ.

So sánh với Mẫu thiết kế

- Framework có phạm vi lớn hơn mẫu thiết kế.
  - Trong khi mẫu thiết kế mô tả một giải pháp cho một vấn đề thiết kế cụ thể,
  - framework là một ứng dụng bán hoàn chỉnh có thể được tùy chỉnh để tạo ra một ứng dụng hoàn chỉnh.
  - Ví dụ:
    - IBM WebSphere và Microsoft .NET là các framework nền tảng triển khai các ứng dụng.
    - Khung web: Express.js (Node.js), Django (Python), Ruby on Rails (Ruby)
- Framework xây dựng bằng cách sử dụng nhiều mẫu thiết kế .
  - framework kết hợp các mẫu thiết kế đã được thiết lập tốt,
    - cung cấp các giải pháp đã được chứng minh cho các vấn đề thiết kế thường gặp.
  - Việc tái sử dụng các mẫu này trong framework
    - giúp tiết kiệm thời gian phát triển và thúc đẩy mã nguồn mạnh mẽ, dễ bảo trì.

Các khía cạnh việc thúc đẩy tái sử dụng

- Tái sử dụng kiến trúc:
  - Các framework cung cấp một thiết kế kiến trúc có sẵn cho nhóm vấn đề cụ thể.
  - thay vì phải thiết kế lại từ đầu cho mỗi ứng dụng mới.
- Lớp trừu tượng và giao diện:
  - Các framework thường bao gồm các lớp trừu tượng và giao diện hành vi cốt lõi và tương tác.
  - thúc đẩy tính đa hình và cho phép các thành phần có thể hoán đổi cho nhau.
- Đảo ngược quyền kiểm soát:
  - nguyên tắc đảo ngược (Nguyên tắc Hollywood: "Đừng gọi cho chúng tôi, chúng tôi sẽ gọi bạn"
  - giúp đơn giản hóa việc tích hợp và nhất quán giữa các ứng dụng trên cùng một framework
- Các mẫu thiết kế:
  - kết hợp các mẫu thiết kế đã được thiết lập tốt, cung cấp các giải pháp đã được chứng minh
- Giải pháp dành riêng cho từng lĩnh vực:
  - thiết kế riêng cho các lĩnh vực ứng dụng cụ thể (ví dụ: web, ứng dụng GUI, truy cập dữ liệu).
- Giảm nỗ lực phát triển:
  - cung cấp đáng kể cấu trúc và chức năng cốt lõi của ứng dụng, giúp giảm lượng mã cần viết,

2.3 Thư viện lớp (Class Libraries)

- Một thư viện lớp (class library)
  - là nền tảng cho thiết kế phần mềm gồm tập hợp các thành phần phần mềm được viết sẵn, thử nghiệm trước và có thể tái sử dụng (lớp, giao diện, v.v.) cho việc kết hợp vào nhiều dự án.
- Nó thúc đẩy việc tái sử dụng bằng cách
  - cung cấp kho lưu trữ chức năng tập trung, chuẩn hóa, cho phép tránh phải viết lại mã chung
- Chiến lược tái sử dụng hiệu quả,
  - các thư viện lớp nên được cân nhắc ngay từ đầu quá trình thiết kế.
  - xem xét các thành phần thư viện có thể tích hợp từ dưới lên
  - thay vì thiết kế toàn bộ từ trên xuống rồi cố gắng tái sử dụng sau.

Đặc điểm

- Thư viện lớp được sử dụng để phát triển các framework và component, cũng như để hỗ trợ các lớp ở tầng Foundation (nền tảng) của kiến trúc hệ thống.
  - Các lớp trong thư viện có tính gắn kết cao và khớp nối lỏng lẻo.
  - Chúng thường được thiết kế để giải quyết các vấn đề chung
  - ví dụ:
    - các lớp để xử lý dữ liệu, giao diện người dùng, kết nối mạng) hơn là các vấn đề cụ thể của một ứng dụng.
- Không giống như framework,
  - thư viện lớp thường không chuyên biệt cho một lĩnh vực cụ thể mà cung cấp các chức năng hỗ trợ cơ bản hơn.

2.4 Components (Thành phần)

- Một component
  - là một phần mềm độc lập, được đóng gói, có thể được cắm vào một hệ thống để cung cấp một tập hợp các chức năng cần thiết.
  - Mỗi component có một giao diện chương trình ứng dụng (API) được xác định rõ ràng, che giấu các chi tiết bên trong của nó.
- Đặc điểm:
  - Các component có thể được coi là các hệ thống con có thể tự triển khai.
  - Chúng có thể được lắp ráp để xây dựng các hệ thống lớn hơn.
  - Chúng được thiết kế để hoạt động trong các môi trường khác nhau và có thể được sử dụng bởi nhiều ứng dụng.
- Lợi ích:
  - Việc sử dụng lại các component làm tăng tốc độ phát triển, giảm chi phí và nâng cao độ tin cậy của hệ thống.

Practical Components

- Individual Classes/Subroutines:
  - Các đơn vị mã cơ bản có thể được thiết kế để thực hiện các chức năng cụ thể, được xác định rõ ràng và có sẵn để tái sử dụng.
- Abstract Data Structures:
  - Cấu trúc dữ liệu chung như hàng đợi hoặc ngăn xếp có thể được triển khai và tái sử dụng trên nhiều ứng dụng khác nhau.
- Configuration Props/Parameters:
  - Cho phép thay đổi hành vi và giao diện thông qua cấu hình thay vì mã lõi
  - giúp các thành phần dễ thích ứng hơn và có thể tái sử dụng rộng rãi hơn.
- Thư viện/Kho lưu trữ thành phần:
  - Nơi tập trung để lưu trữ, truy cập và quản lý các thành phần có thể tái sử dụng, bao gồm tài liệu, giúp tìm và sử dụng đúng các tài sản hiện có.

- Tóm lại, việc xác định cơ hội tái sử dụng là một hoạt động thiết kế chiến lược,
  - giúp tận dụng các giải pháp đã có để xây dựng hệ thống nhanh hơn, đáng tin cậy hơn và dễ bảo trì hơn.
- Trong giai đoạn thiết kế,
  - tái sử dụng không chỉ giới hạn ở việc sử dụng lại mã nguồn
  - mà còn bao gồm cả việc tái sử dụng các thiết kế đã được kiểm chứng.
- Nó yêu cầu nhà phân tích phải xem xét các
  - mẫu thiết kế, framework, thư viện lớp và component có sẵn để áp dụng vào thiết kế của mình.

Hoạt động 3: Tái cấu trúc Thiết kế (Restructuring)

Hoạt động 3: Tái cấu trúc Thiết kế (Restructuring)

- "Thiết kế không phải là một quá trình một chiều.
  - Đôi khi, chúng ta cần phải nhìn lại và 'dọn dẹp' lại cấu trúc của mình.
- Hoạt động 'Tái cấu trúc'
  - cải thiện thiết kế, làm đơn giản và dễ bảo trì ,
  - ngay cả khi không thay đổi hành vi bên ngoài.
- Tái cấu trúc giúp
  - tăng Cohesion và giảm Coupling, dễ hiểu và dễ bảo trì hơn trong dài hạn.
- Kỹ thuật chính:
  1. Factoring: Tách một lớp/phương thức lớn thành các lớp/phương thức nhỏ hơn, gắn kết hơn.
  2. Normalization: Áp dụng các quy tắc chuẩn hóa để xác định các lớp còn thiếu và giảm sự dư thừa dữ liệu.

1. Factoring (Tách nhân tố)

- Factoring
  - là quá trình tách một module (ví dụ: một lớp hoặc phương thức) thành các module độc lập.
  - Kỹ thuật này giúp giảm sự trùng lặp và tăng tính tái sử dụng do các điểm chung được tách ra.
- Đơn giản hóa thiết kế:
  - tách các phần phức tạp, thiết kế tổng thể trở nên dễ hiểu và dễ quản lý.
- Tăng tính tái sử dụng (Reuse):
  - Khi các thuộc tính (attributes) và phương thức (methods) tương tự nhau được tìm thấy trong tập các lớp,
    - chúng có thể được "tách" ra và đặt vào một lớp mới để dùng chung
    - Lớp mới được liên kết với các lớp ban đầu thông qua kế thừa (generalization) hoặc tập hợp (aggregation), giúp tái sử dụng mã nguồn hiệu quả.

Vd Factoring

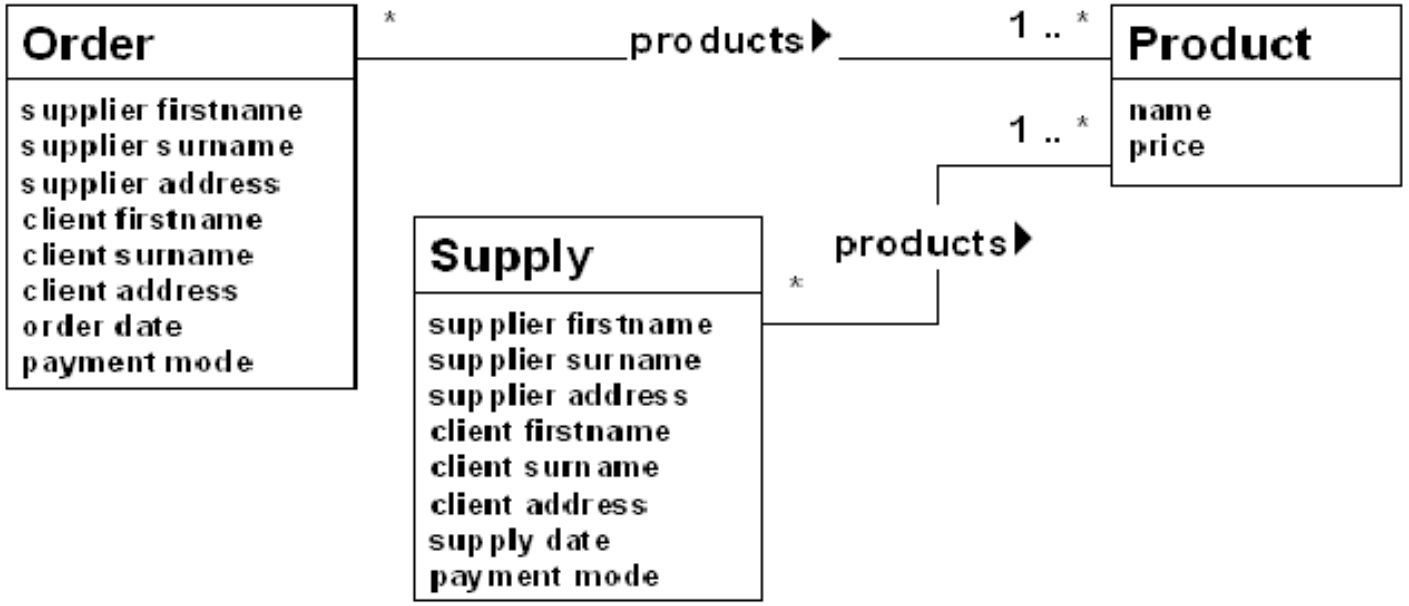
- Ví dụ:
  - các lớp Nurse (Y tá), Receptionist (Lễ tân), và Doctor (Bác sĩ) đều có các thuộc tính và phương thức giống nhau như name, address, phone, dateOfBirth.
- Thay vì định nghĩa các thuộc tính và phương thức này lặp đi lặp lại trong mỗi lớp, thực hiện factoring bằng cách
  - tạo ra một lớp cha mới là Employee (Nhân viên) để chứa các điểm chung đó.
  - Sau đó, các lớp Nurse, Receptionist, và Doctor sẽ kế thừa từ lớp Employee.
- Quá trình này cũng liên quan chặt chẽ đến
  - Abstraction (trừu tượng hóa - tạo ra một ý tưởng cấp cao hơn từ một tập hợp các ý tưởng) và
  - Refinement (tinh chỉnh - xác định các lớp con chi tiết hơn từ một lớp chung).

2. Normalization (Chuẩn hóa)

- thường dùng trong chuẩn hóa cơ sở dữ liệu quan hệ
- Normalization trong OOAD
  - là một kỹ thuật giúp xác định các lớp còn thiếu trong thiết kế và đảm bảo mỗi lớp có một mục đích duy nhất
- Mục đích:
  - Phát hiện các lớp bị thiếu:
    - Bằng cách áp dụng các quy tắc chuẩn hóa, nhà phân tích có thể nhận ra rằng một số thuộc tính không thực sự thuộc về một lớp hiện có mà nên được đặt trong một lớp mới chưa được xác định.
  - Cải thiện tính gắn kết (Cohesion):
    - Chuẩn hóa giúp đảm bảo rằng một lớp chỉ đại diện cho một khái niệm duy nhất, từ đó tăng tính gắn kết của lớp đó.

Ví dụ:

- Trong quá trình xem xét lớp Order,
- nếu có các thuộc tính lặp lại liên quan đến sản phẩm (như productID_1, productName_1, quantity_1, productID_2...),
- việc áp dụng chuẩn hóa sẽ dẫn đến việc tạo ra một lớp mới là Product và
- một lớp liên kết OrderLine (hoặc ProductOrder) để quản lý mối quan hệ nhiều-nhiều giữa Order và Product.



3. Chuyển đổi các Mối quan hệ thành Thuộc tính

- (Converting Relationships to Attributes)
  - Một phần quan trọng của việc tái cấu trúc thiết kế là chuẩn bị cho việc triển khai.
  - Hầu hết các ngôn ngữ lập trình hướng đối tượng không phân biệt trực tiếp giữa thuộc tính và các mối quan hệ (association, aggregation).
  - Do đó, tất cả các mối quan hệ này phải được chuyển đổi thành các thuộc tính trong các lớp tương ứng để có thể lập trình được.

- Ví dụ:
  - Một mối quan hệ một-nhiều giữa lớp Customer và lớp Order (một khách hàng có thể có nhiều đơn hàng) sẽ được triển khai bằng cách:
    - Thêm một thuộc tính vào lớp Customer để tham chiếu đến một tập hợp (collection) các đối tượng Order.
    - Thêm một thuộc tính vào lớp Order để tham chiếu ngược lại đối tượng Customer sở hữu nó.

- Hoạt động này diễn ra sau khi các đặc tả đã được bổ sung và các cơ hội tái sử dụng đã được xác định.
- Mục tiêu của Tái cấu trúc là xem xét lại và cải thiện mô hình thiết kế đang phát triển để đảm bảo nó có chất lượng cao, dễ bảo trì và tối ưu hóa.
- Đây là một bước quan trọng để tinh chỉnh thiết kế trước khi chuyển sang giai đoạn tối ưu hóa và triển khai.
- Tóm lại, Tái cấu trúc Thiết kế (Restructuring) là một hoạt động thiết yếu để cải tiến và làm sạch mô hình thiết kế.
- Nó sử dụng các kỹ thuật như Factoring và Normalization để đảm bảo thiết kế có cấu trúc tốt, có tính gắn kết cao, khớp nối lỏng lẻo, và sẵn sàng cho việc triển khai bằng cách chuyển đổi các mối quan hệ logic thành các thuộc tính cụ thể.

Hoạt động 4:
Tối ưu hóa Thiết kế (Optimizing)

Hoạt động 4: Tối ưu hóa Thiết kế (Optimizing)

- "Một bản thiết kế tốt không chỉ cần dễ hiểu mà còn phải hiệu quả về mặt hiệu năng.
- 'Tối ưu hóa' là quá trình cân bằng giữa hai yếu tố này.
  - Tuy nhiên, việc tối ưu hóa giống như 'con dao hai lưỡi', nó có thể cải thiện hiệu suất nhưng cũng có thể làm tăng sự phức tạp.
- Slide này sẽ trình bày các kỹ thuật tối ưu hóa và những lưu ý quan trọng khi thực hiện."
- Các kỹ thuật tối ưu hóa:
  - Rà soát đường dẫn truy cập,
  - giảm độ phức tạp của phương thức,
  - rút gọn thuật toán,
  - sáp nhập các lớp có mối quan hệ 1-1
  - Lưu ý: Việc tối ưu hóa thường làm tăng coupling và giảm cohesion, cần được thực hiện cẩn trọng.

- Xem xét các đường dẫn truy cập giữa các đối tượng (Review Access Paths):
- ◦ Vấn đề: Đôi khi, một thông điệp từ một đối tượng này đến đối tượng khác phải đi qua một con đường dài, tức là nó phải đi qua nhiều đối tượng trung gian. Nếu đường dẫn này dài và được sử dụng thường xuyên, nó có thể làm giảm đáng kể hiệu suất của hệ thống.
- ◦ Giải pháp: Để giải quyết vấn đề này, bạn nên xem xét việc thêm một đường dẫn dự phòng (redundant path). Điều này có thể được thực hiện bằng cách thêm một thuộc tính (attribute) vào đối tượng gọi, thuộc tính này sẽ lưu trữ một kết nối trực tiếp đến đối tượng ở cuối đường dẫn, giúp giảm thiểu thời gian truy cập.

- 2. Xem xét các thuộc tính của mỗi lớp (Review Attributes):
- ◦ Vấn đề: Một thuộc tính có thể được đặt trong một lớp nhưng lại chủ yếu được sử dụng bởi các phương thức của một lớp khác. Nếu các phương thức duy nhất sử dụng một thuộc tính là các phương thức đọc và cập nhật, và chỉ các thể hiện của một lớp duy nhất gửi thông điệp để đọc và cập nhật thuộc tính đó, thì thuộc tính đó có thể đang ở sai lớp.
- ◦ Giải pháp: Di chuyển thuộc tính đó sang lớp gọi sẽ giúp tăng tốc độ hệ thống một cách đáng kể.

- 3. Tránh tính toán lại bằng cách sử dụng thuộc tính dẫn xuất (Derived Attributes):
- ◦ Vấn đề: Một số giá trị hoặc kết quả có thể yêu cầu các phép tính phức tạp và thường xuyên được truy cập. Việc tính toán lại các giá trị này mỗi khi cần có thể rất tốn kém về mặt hiệu suất.
- ◦ Giải pháp: Thay vì tính toán lại, hãy tạo một thuộc tính dẫn xuất (derived attribute) hoặc sử dụng kỹ thuật lưu trữ kết quả tính toán (caching).
- ▪ Một trigger (trình kích hoạt) có thể được thêm vào các thuộc tính cơ sở (những thuộc tính được dùng trong phép tính). Khi một trong các thuộc tính cơ sở thay đổi, phép tính sẽ được thực hiện lại và giá trị của thuộc tính dẫn xuất được cập nhật.
- ▪ Một cách tiếp cận khác là trì hoãn việc tính toán lại cho đến lần tiếp theo thuộc tính dẫn xuất được truy cập, giúp đảm bảo rằng phép

- 4. Xem xét thứ tự thực thi của các câu lệnh (Review Execution Order):
- ◦ Vấn đề: Trong các phương thức được sử dụng thường xuyên, thứ tự thực thi của các câu lệnh có thể ảnh hưởng đến hiệu suất.
- ◦ Giải pháp: Trong một số trường hợp, việc sắp xếp lại các câu lệnh có thể giúp hệ thống hoạt động hiệu quả hơn. Ví dụ, nếu một quy trình tìm kiếm có thể được thu hẹp bằng cách tìm kiếm trên một thuộc tính trước một thuộc tính khác, thì thuật toán tìm kiếm nên được tối ưu hóa để luôn tìm kiếm theo thứ tự đã xác định trước đó.

- 5. Sáp nhập các lớp có quan hệ 1-1 (Merging 1-1 Relationships):
- ◦ Vấn đề: Các đối tượng tham gia vào mối quan hệ một-một (one-to-one association) mà cả hai đều phải tồn tại để một trong hai tồn tại có thể làm tăng sự phức tạp không cần thiết.
- ◦ Giải pháp: Để tăng hiệu quả, bạn có thể xem xét việc sáp nhập hai lớp này thành một lớp duy nhất. Tuy nhiên, cần lưu ý rằng tùy thuộc vào định dạng lưu trữ đối tượng được chọn (ví dụ: cơ sở dữ liệu quan hệ), việc giữ hai lớp riêng biệt có thể lại hiệu quả hơn trên tầng quản lý dữ liệu (data management layer).

- Một thiết kế tốt phải quản lý được sự đánh đổi không thể tránh khỏi giữa các yếu tố.
- Cho đến thời điểm này, trọng tâm của quá trình thiết kế là phát triển một thiết kế dễ hiểu.
- Tuy nhiên, việc tăng tính dễ hiểu thường tạo ra một thiết kế không hiệu quả.
- Ngược lại, việc tập trung vào các vấn đề hiệu quả sẽ tạo ra một thiết kế khó hiểu hơn.
- Do đó, tối ưu hóa là quá trình điều chỉnh thiết kế một cách có chủ đích để cải thiện hiệu quả xử lý và đáp ứng các yêu cầu phi chức năng (nonfunctional requirements)
- Tóm lại, Tối ưu hóa Thiết kế là một hoạt động quan trọng để cân bằng giữa tính dễ hiểu và hiệu quả của hệ thống.
- Nó đòi hỏi nhà phân tích phải xem xét cẩn thận các đường dẫn truy cập, vị trí của các thuộc tính, các phép tính lặp lại và cấu trúc của các lớp để đảm bảo hệ thống đáp ứng được cả yêu cầu chức năng và phi chức năng về hiệu suất.

Method Specification

Tại sao cần Đặc tả Phương thức? Cầu nối giữa Yêu cầu và Mã nguồn

- Từ Thiết kế đến Mã nguồn: khoảng trống "Mơ hồ"
- Giai đoạn phân tích và thiết kế ban đầu tạo ra các mô hình ở mức cao (high-level models).
  - Tuy nhiên, các mô hình này không đủ chi tiết để lập trình viên có thể viết mã một cách chính xác.
- Đặc tả Phương thức
  - là cầu nối thiết yếu để biến các quyết định thiết kế thành những chỉ dẫn rõ ràng, không mơ hồ,
  - đảm bảo rằng mã nguồn được xây dựng đúng ngay từ đầu.

Đặc tả Phương thức (Method Specification)

- Vấn đề:
  - Các mô hình phân tích(sơ đồ lớp, sơ đồ tuần tự) cho thấy cái gì cần làm, nhưng không chỉ rõ làm như thế nào ở mức độ chi tiết.
  - Nếu không có chỉ dẫn cụ thể, mỗi lập trình viên sẽ tự diễn giải yêu cầu theo cách riêng, dẫn đến một hệ thống không nhất quán và đầy lỗi.
- Giải pháp: Đặc tả Phương thức
  - là một tài liệu bằng văn bản cung cấp các chỉ dẫn rõ ràng về cách viết mã để triển khai một phương thức.
- Mục tiêu:
  - Cung cấp cho lập trình viên đủ thông tin để xây dựng hệ thống mà không cần phải đoán mò hay đưa ra các giả định sai lầm, giúp giảm thiểu việc làm lại và sửa lỗi tốn kém.

Hợp đồng (Contracts)

- Nền tảng của Đặc tả
  - Hợp đồng (Contracts) giữa các Đối tượng
  - Thiết lập "Luật chơi": Ai làm gì và Chịu trách nhiệm ra sao?
- "Trong thế giới phần mềm, các đối tượng cũng cần 'hợp đồng' để làm việc với nhau một cách hiệu quả.
- Trước khi đi vào chi tiết logic, chúng ta cần xác định rõ trách nhiệm và sự tương tác giữa các thành phần của hệ thống.
- Hợp đồng (Contract)
  - là một khái niệm được áp dụng vào thiết kế phần mềm,
  - giúp hình thức hóa các tương tác này một cách tường minh, tạo nền tảng vững chắc cho việc đặc tả phương thức.

thành phần của một Hợp đồng

- Hợp đồng là một thỏa thuận giữa đối tượng Client (bên yêu cầu dịch vụ) và đối tượng Server (bên cung cấp dịch vụ).
- Mô hình hóa trách nhiệm:
  - Hợp đồng chỉ định rõ những ràng buộc mà Client phải đáp ứng (tiền điều kiện) và những gì Server cam kết sẽ thực hiện (hậu điều kiện).
  - Nó đảm bảo cả hai bên đều hiểu rõ trách nhiệm của mình..“
- Các thành phần của một Hợp đồng bao gồm:
  1. Tên phương thức và lớp chịu trách nhiệm.
  2. Mô tả trách nhiệm của phương thức.
  3. Các đối số (arguments) nhận vào và giá trị trả về (return value).
  4. Các ràng buộc: (Preconditions) và (Postconditions)

Ví dụ về Hợp đồng

- Ví dụ về mẫu hợp đồng cho phương thức addOrder trong lớp Customer

Method Name:	`addOrder`	**Class Name:**	`Customer`
Clients (Consumers):	`Order` class	**ID:**	36
Associated Use Cases:	`addCustomerOrder`		
Description:	Thêm một đơn hàng mới cho khách hàng, giữ danh sách đơn hàng được sắp xếp.		
Arguments Received:	`anOrder:Order`	**Value Returned:**	`void`
Pre-Conditions:	Đơn hàng chưa tồn tại trong danh sách: `not orders.includes(anOrder)`		
Post-Conditions:	Danh sách đơn hàng mới bao gồm đơn hàng vừa thêm: `Order=Orders@pre.including(anOrder)`		

Ràng buộc (Constraints) và Ngôn ngữ OCL

- Ràng buộc: Một quy tắc hoặc điều kiện phải luôn đúng.
- Các loại ràng buộc chính:
  - Pre-condition (Tiền điều kiện).
  - Post-condition (Hậu điều kiện).
  - Invariant (Bất biến).

Ràng buộc bằng OCL

- OCL (Object Constraint Language):
  - Một ngôn ngữ hình thức để đặc tả các ràng buộc một cách chính xác, không mơ hồ.
  - Một biểu thức OCL là một câu lệnh khai báo và luôn trả về giá trị đúng (true) hoặc sai (false).
  - Ví dụ: self.balance >= 0 hoặc self.age > 65 or self.customerType = "prime".
- cách sử dụng các loại ràng buộc và OCL để đảm bảo hệ thống hoạt động đúng theo logic đã định.
  - Lớp: Account
  - Phương thức: withdraw(amount: double)
  - Tiền điều kiện:
    - OCL: self.balance >= amount.
    - Diễn giải: Số dư tài khoản phải lớn hơn hoặc bằng số tiền rút.
  - Hậu điều kiện:
    - OCL: self.balance = self.balance@pre - amount.
    - Diễn giải: Số dư mới bằng số dư cũ (@pre) trừ đi số tiền đã rút.
  - Bất biến:
    - OCL: self.balance >= 0.
    - Diễn giải: Số dư tài khoản không bao giờ được âm.

Từ "Cái gì" đến "Cách thức"

- Hợp đồng và ràng buộc đã định nghĩa "cái gì" (what) một phương thức phải làm.
- Bây giờ, chúng ta cần mô tả "cách thức" (how) nó thực hiện logic của nó.
- Đây là giai đoạn tạo ra Đặc tả Phương thức (Method Specification).
  - đánh dấu một bước chuyển quan trọng trong quy trình thiết kế, từ việc xác định yêu cầu sang việc mô tả giải pháp.
  - Đặc tả Phương thức như là công cụ để thực hiện bước chuyển đổi này, tạo ra hướng dẫn chi tiết cho lập trình viên.

Đặc tả Phương thức (Method Specification)

- Method Specification
  - Là một tài liệu mô tả chi tiết logic bên trong một phương thức.
- một bản đặc tả hoàn chỉnh.
  - bản đặc tả cần phải đầy đủ và có cấu trúc.
  - những thông tin cần thiết để tạo ra một tài liệu hướng dẫn rõ ràng cho lập trình viên, mô tả mọi thứ họ cần biết để viết mã
  - trong đó phần quan trọng nhất là Đặc tả thuật toán.
- Không có định dạng chung cho việc này
  - mẫu đặc tả tiêu chuẩn đảm bảo rằng tất cả thông tin cần thiết đều được cung cấp một cách nhất quán.

Các Thành phần của một Đặc tả Phương thức

- Một mẫu đặc tả phương thức (Method Specification Form) thường bao gồm bốn thành phần chính:
  - 1. Thông tin chung (General Information):
    - Tên phương thức, tên lớp, ID, lập trình viên được giao, ngày đến hạn. Giúp quản lý và theo dõi công việc lập trình.
  - 2. Sự kiện (Events):
    - Mô tả các sự kiện kích hoạt phương thức (ví dụ: người dùng nhấp chuột, một sự kiện hệ thống). Điều này đặc biệt quan trọng đối với các chương trình hướng sự kiện (event-driven programs).
  - 3. Truyền thông điệp (Message Passing):
    - Liệt kê các đối số (arguments) nhận vào và giá trị trả về. Thông tin này được lấy từ các sơ đồ tương tác (interaction diagrams).
  - 4. Đặc tả Thuật toán (Algorithm Specification):
    - Mô tả logic chi tiết bên trong phương thức.

Công cụ Đặc tả Thuật toán

- Phần 'trái tim' của một đặc tả phương thức chính là đặc tả thuật toán, nơi chúng ta mô tả logic xử lý.
- Đặc tả thuật toán.
  - Mô tả luồng hoạt động của một quy trình nghiệp vụ hoặc logic chi tiết của một phương thức.
  - có nhiều cách để diễn đạt logic và có thể lựa chọn công cụ phù hợp nhất với độ phức tạp của phương thức.
- 2 công cụ phổ biến để mô tả logic của một phương thức:
  - 1. Structured English (Tiếng Anh có cấu trúc):
    - Sử dụng các cấu trúc lệnh đơn giản (IF-THEN-ELSE, REPEAT-UNTIL) để mô tả các bước một cách rõ ràng.
  - 2. Activity Diagrams (Sơ đồ hoạt động):
    - Một công cụ của UML, hiệu quả để trực quan hóa các thuật toán phức tạp.

Structured English: Logic cho Cả Người nghiệp vụ và Lập trình viên

- Khi các ràng buộc đã rõ ràng, chúng ta cần đi sâu vào "công thức" xử lý bên trong mỗi phương thức.
- Structured English (Tiếng Anh có cấu trúc)
  - Là một cách viết hình thức, sử dụng các câu ngắn gọn và từ vựng giới hạn để mô tả các bước của một quy trình.
  - là một công cụ mạnh mẽ, giúp diễn đạt logic thuật toán bằng một dạng ngôn ngữ tự nhiên có cấu trúc, dễ hiểu cho mọi bên liên quan.
- Ưu điểm:
  - Loại bỏ sự mơ hồ của ngôn ngữ tự nhiên nhưng vẫn đủ đơn giản để người dùng nghiệp vụ có thể đọc và xác nhận tính đúng đắn của logic.

Đặc tả bằng Structured English

- Dựa trên 3 cấu trúc logic cơ bản:
  - 1. Tuần tự (Sequence):
    - Thực hiện các bước theo thứ tự.
  - 2. Lựa chọn (Selection):
    - Sử dụng các cấu trúc như IF...THEN...ELSE...ENDIF.
  - 3. Lặp (Iteration):
    - Sử dụng các cấu trúc như REPEAT...UNTIL hoặc FOR....
- Ví dụ:
  - Method: Calculate_Total_Order_Cost
  - Description:
    - Phương pháp này tính tổng chi phí đơn hàng của khách hàng
  - Inputs:
    - customer_id: Mã định danh duy nhất cho khách hàng.
    - items_list: Danh sách các mặt hàng trong đơn đặt hàng, mỗi mặt hàng có số lượng và đơn giá.
  - Process:

- Process:

INITIALIZE: subtotal to 0.
FOR EACH: item in items_list:
    CALCULATE: item_cost = item.quantity * item.unit_price.
    ADD: item_cost to subtotal.
IF: subtotal is greater than or equal to 50:
    SET: shipping_cost to 0 (free shipping).
ELSE:
    SET: shipping_cost to 5.
CALCULATE: total_cost = subtotal + shipping_cost.
RETURN: total_cost.
Output:
total_cost: Chi phí tính toán cuối cùng của đơn hàng.

Ví dụ: Đặc tả bằng Sơ đồ Hoạt động

- Phương thức: middleListInsert()
  - Mô tả:
    - Chèn một node mới vào giữa danh sách liên kết đã được sắp xếp
  - Sơ đồ này trực quan hóa các bước:
    - tìm vị trí chèn,
    - tạo node mới, và
    - cập nhật các con trỏ để chèn node vào danh sách.
  - Slide này minh họa việc sử dụng Sơ đồ hoạt động, một công cụ trực quan của UML.
- ưu điểm của sơ đồ trong việc mô tả các luồng xử lý phức tạp, đặc biệt là các điểm quyết định và vòng lặp.

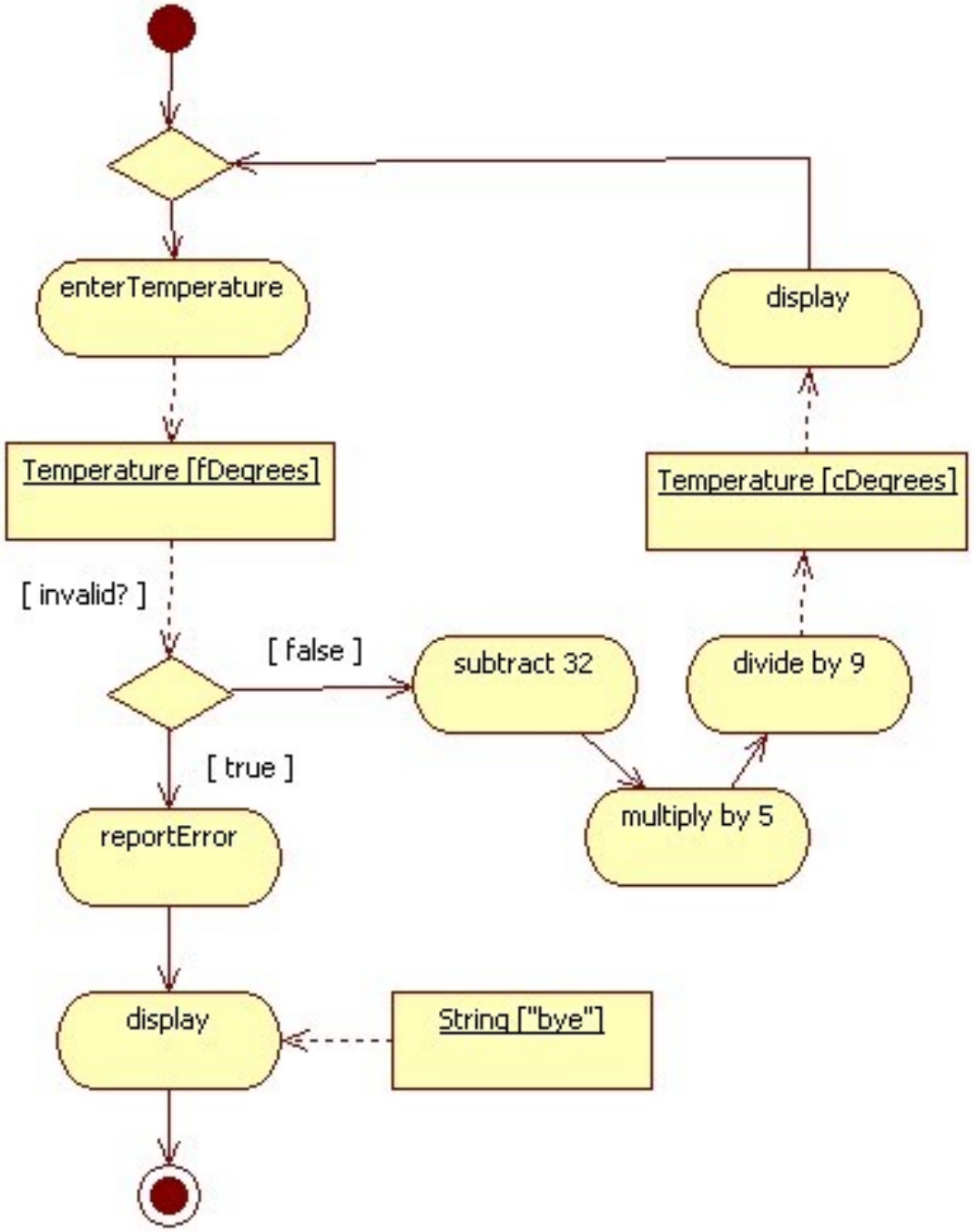
```
Create newOrderNode with the newOrder
 IF emptyList?()
THEN
        FirstNode = LastNode = newOrderNode
ELSE IF newOrderNode < FirstNode
THEN
        newOrderNode.setNextNode(FirstNode
)
        FirstNode = newOrderNode
ELSE IF newOrderNode > LastNode THEN
  ... ELSE
          middleListInsert(newOrderNode)
ENDIF
```

Sơ đồ hoạt động (Activity Diagrams) cho Phương thức

- "Khi logic trở nên phức tạp với nhiều nhánh rẽ và luồng song song,
- 'Sơ đồ hoạt động' là công cụ trực quan hóa tuyệt vời.
  - Nó giống như một bản đồ chi tiết chỉ dẫn luồng công việc của một phương thức.
- Các thành phần chính:
  - Hành động/Hoạt động (Action/Activity): Các bước xử lý.
  - Nút Quyết định (Decision Node): Thể hiện các nhánh rẽ.
  - Luồng điều khiển (Control Flow): Trình tự thực hiện.
- Cảnh báo:
  - Nếu một phương thức cần đến một biểu đồ hoạt động phức tạp để mô tả, đó có thể là dấu hiệu cho thấy phương thức đó nên được chia nhỏ hơn (để tăng tính gắn kết - cohesion) hoặc có thể thiếu một lớp nào đó trong thiết kế.

Ví dụ: Đặc tả bằng Sơ đồ Hoạt động

- Phương thức: FtoC()
  - Mô tả:
    - Đổi từ độ F sang độ C
  - Sơ đồ này trực quan hóa các bước:
    - Nhập nhiệt độ
    - Kiểm tra
    - Chuyển đổi -35 x5 /9
    - Hiển thị.



Từ phân tích đến mã nguồn

- Sơ đồ Lớp Thiết kế (Design Class Diagram - DCD)
  - sản phẩm đầu ra quan trọng của việc thiết kế lớp
  - sự mở rộng của Sơ đồ Lớp Miền vấn đề,
  - bổ sung các chi tiết kỹ thuật cần thiết cho việc lập trình.
  - một danh sách kiểm tra cụ thể về những thông tin kỹ thuật cần bổ sung vào sơ đồ lớp phân tích, biến nó thành một bản vẽ chi tiết sẵn sàng cho việc lập trình.
- kết nối tất cả các khái niệm một quy trình liền mạch.
  - thấy được mối liên hệ trực tiếp từ mô hình phân tích, qua các bước thiết kế chi tiết, và cuối cùng là ánh xạ sang mã nguồn thực tế,

Áp dụng các Công cụ vào Thực tế:Từ Đặc tả đến Mã nguồn

- ví dụ đặc tả phương thức insertOrder.
  - Quá trình này minh họa cách một thiết kế chi tiết được chuyển thành các chỉ dẫn lập trình cụ thể.
  - Thông tin chung:
    - Tên phương thức: insertOrder,
    - Lớp: OrderList,
    - Lập trình viên: J. Doe.
  - Truyền thông điệp:
    - Nhận vào: anOrder (kiểu Order).
    - Gửi đi:
    - OrderNode.new(),
    - Order.getOrderNumber(), v.v..

Vd minh họa toàn bộ quá trình "hiện thực hóa" một yêu cầu.

- 1. Lớp Phân tích:
  - Customer (name, address)
- 2. Lớp Thiết kế (DCD):
  - || «entity» Customer ||
    - - name: string
    - - address: string
  - + updateAddress(newAddress: string): void
- 3. Đặc tả phương thức
  - updateAddress (Pseudocode):
- 4. Mã nguồn (Java):

```
public class Customer {
    private String name;
    private Address address;

    public void updateAddress(Address
newAddress) {
        // Logic triển khai từ đặc tả phương
thức
        if (newAddress != null) {
            this.address = newAddress;
        }
    }
    // Các phương thức khác...
}
```

Xác minh và Xác thực Thiết kế

Xác minh và Xác thực (Verification and Validation) Thiết kế

- "Sau khi hoàn thành bản thiết kế, làm sao để chắc chắn rằng nó đúng?
- Đó là lúc 'Xác minh và Xác thực' vào cuộc.
  - Đây là quá trình kiểm tra chéo để đảm bảo thiết kế của chúng ta chính xác, đầy đủ và đáp ứng yêu cầu.
- Mục đích:
  - Đảm bảo thiết kế chính xác, đầy đủ và đáp ứng yêu cầu.
  - Sửa lỗi ở giai đoạn thiết kế rẻ hơn rất nhiều so với sửa lỗi trong mã nguồn.
- Quá trình này cần được thực hiện lặp đi lặp lại sau mỗi thay đổi.
- Các kỹ thuật chính:
  - Walkthroughs (Rà soát),
  - Role-playing (Đóng vai).

Kỹ thuật Walkthrough trong Xác thực Thiết kế

- "Kỹ thuật 'Walkthrough' giống như một buổi họp rà soát, nơi cả nhóm cùng nhau xem xét các mô hình thiết kế.
- Mục tiêu là để tìm ra sự thiếu nhất quán giữa các mô hình chức năng, cấu trúc và hành vi.
- Bằng cách 'cân bằng' các mô hình này, chúng ta đảm bảo rằng toàn bộ thiết kế là một thể thống nhất và hợp lý."
- **Nội dung:**
  - Rà soát tất cả các mô hình để đảm bảo tính nhất quán.
  - Kiểm tra sự cân bằng giữa các mô hình (Balancing Models):
  - Functional vs. Structural,
  - Functional vs. Behavioral,
  - Structural vs. Behavioral.

Kỹ thuật Role-Playing trong Xác thực Thiết kế

- "Kỹ thuật 'Role-Playing' là một cách xác thực sinh động.
- Chúng ta sẽ 'đóng vai' các đối tượng và thực thi các kịch bản của use case.
- Việc này giúp chúng ta kiểm tra xem các ràng buộc, hợp đồng và logic trong đặc tả phương thức có hoạt động đúng như mong đợi trong thực tế hay không.
- Đây là cách 'chạy thử' bản thiết kế trước khi viết mã."

- **Nội dung:**
  - Thực thi các kịch bản của use case bằng cách đóng vai các đối tượng.
  - Kiểm tra các ràng buộc: Invariants, Pre-conditions, và Post-conditions.
  - Kiểm tra logic trong các đặc tả phương thức có thực thi đúng không.

Liên kết với các Mô hình khác: Đảm bảo sự Nhất quán

- Việc tạo ra các đặc tả riêng lẻ là chưa đủ.
  - Chúng phải nhất quán với các mô hình khác đã được tạo ra trong quá trình phân tích và thiết kế để tạo thành một bức tranh tổng thể chính xác về hệ thống.
- Đặc tả Phương thức và Sơ đồ Lớp (Class Diagram):
  - Mỗi phương thức được đặc tả phải tồn tại trong một lớp trên sơ đồ lớp thiết kế.
  - Các thuộc tính được sử dụng trong logic phải được định nghĩa trong các lớp liên quan.

- Đặc tả Phương thức và Sơ đồ Tương tác (Interaction Diagrams):
  - Các thông điệp (messages) được gửi và nhận trong Đặc tả Phương thức phải khớp với các thông điệp trên Sơ đồ Tuần tự (Sequence Diagram) và Sơ đồ Giao tiếp (Communication Diagram).
- Các đối số (arguments) và giá trị trả về cũng phải nhất quán.
- Đặc tả Phương thức và Hợp đồng (Contracts):
  - Các tiền điều kiện và hậu điều kiện trong Đặc tả Phương thức phải tuân thủ nghiêm ngặt các ràng buộc đã được định nghĩa trong Hợp đồng.

Các Lỗi thường gặp trong Thiết kế Lớp và Phương thức

- 'cạm bẫy' thường gặp trong thiết kế lớp và phương thức,
  - tạo ra các lớp quá phụ thuộc,
  - các lớp làm quá nhiều việc,
  - bỏ qua các yêu cầu phi chức năng.
- Nội dung:
  - High Coupling, Low Cohesion.
  - Bỏ qua Non-functional requirements.
  - Tái sử dụng không đúng cách.
  - Thiết kế không đầy đủ, đặc tả mơ hồ.

Sản phẩm cuối cùng của Pha Thiết kế

- Sau khi hoàn thành các hoạt động thiết kế, chúng ta có một bộ tài liệu hoàn chỉnh, sẵn sàng cho việc xây dựng hệ thống:
- Đặc tả Thiết kế Hệ thống (System Design Specification), bao gồm:
  1. Sơ đồ Lớp Thiết kế (Design Class Diagrams) chi tiết.
  2. Sơ đồ Tương tác (Interaction Diagrams) cho mỗi use case.
  3. Sơ đồ Trạng thái (State Machine Diagrams) cho các lớp phức tạp.
  4. Đặc tả Phương thức (Method Specifications) cho các phương thức quan trọng.
  5. Thiết kế CSDL, Giao diện người dùng và Kiến trúc vật lý.
- tổng hợp lại các sản phẩm đầu ra của toàn bộ pha Thiết kế.
  - có một cái nhìn tổng quan về những gì cần tạo ra trong một dự án thực tế.
  - Nó đóng vai trò như một danh sách kiểm tra cuối cùng và cho thấy sự hoàn thiện của quá trình thiết kế.

Tổng kết

- Pha thiết kế chuyển đổi các yêu cầu thành một bản thiết kế kỹ thuật chi tiết.
- Trọng tâm là thiết kế các lớp và phương thức dựa trên các tiêu chí chất lượng như coupling thấp và cohesion cao.
- Các mô hình và tài liệu thiết kế là công cụ giao tiếp quan trọng giữa nhà phân tích và lập trình viên.
- Xác minh và xác thực là hoạt động liên tục để đảm bảo chất lượng.
- Bước tiếp theo: Pha Xây dựng (Construction), nơi các lập trình viên sẽ hiện thực hóa bản thiết kế này thành mã nguồn.


HUST
hust.edu.vn
fb.com/dhbkhn